Quantstamp

DRE

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Cross-Chain Stablecoin and Vault
Timeline	2026-02-23 through 2026-03-02
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• https://github.com/dre-labs/dreusd #e167c45
Auditors	• Rabib Islam Senior Auditing Engineer • Jennifer Wu Auditing Engineer • Cameron Biniamow Auditing Engineer

Documentation quality	High
Test quality	Medium
Total Findings	13 Fixed: 11 Acknowledged: 2
High severity findings	0
Medium severity findings	1 Fixed: 1
Low severity findings	10 Fixed: 8 Acknowledged: 2
Undetermined severity findings	0
Informational findings	2 Fixed: 2

Summary of Findings

The DRE codebase is centered around `dreUSD`, an ERC-20 stablecoin. The `dreUSDManager` is the core orchestrator, handling the minting of `dreUSD` from other stablecoins and managing a dual-queue withdrawal system where user positions are represented as transferable NFTs (`dreWithdrawalNFT`). For yield, users can stake their tokens in the `dreUSDs` ERC-4626 vault, which accrues rewards streamed from the `dreRewardsDistributor`. Price integrity for all conversions is maintained by the `dreUSDOracle`, which uses Chainlink feeds for validation.

The system's architecture is modular and security-focused, incorporating compliance features directly into the `dreUSD` token through sanctions list checks and an address freezing mechanism. It is designed for extensibility, with components like the `dreAaveAdapter` enabling withdrawals to be sourced from external liquidity pools such as Aave. The protocol also supports cross-chain functionality via LayerZero OFT (Omnichain Fungible Token) contracts, including `dreShareOFT` and `dreOVaultComposer`, which allow for the bridging of vault shares. All contracts are upgradeable and managed through a detailed role-based access control system.

One issue concerns scenarios where the transfer of vested rewards to the vault contract results in desynchronization of the vault's asset counter.

A separate issue concerns how transfer of ownership in `dreUSD` may lead to the ownership structure breaking unexpectedly.

We recommend all issues be addressed.

Fix-Review Update 2026-03-18:
The majority of issues have been fixed, with the remaining two being acknowledged.

ID	DESCRIPTION	SEVERITY	STATUS
DRE-1	**dreUSD Manager Can Deflate Share Price During Reward Top-Up**	• Low	Fixed
DRE-2	`dreUSD` **Ownership Can only Be Transferred Once**	• Medium	Fixed
DRE-3	**Cross-Chain Messages Stuck in LayerZero Queue**	• Low	Fixed
DRE-4	**Excessive Access Controls for Sensitive Functionality**	• Low	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
DRE-5	Unclaimable Donated dreUSD	Low	Fixed
DRE-6	Spoke Share Compliance Can Drift From Local `dreUSD` Policy	Low	Fixed
DRE-7	Bridge Sends Can Credit Blocked Recipients on Destination	Low	Acknowledged
DRE-8	Distributor Rotation without Settlement Can Understate Vault Assets	Low	Fixed
DRE-9	`mintAndStake()` Lacks `minSharesOut` Protection	Low	Fixed
DRE-10	Aave Adapter Withdrawals Are Not Bound to Requested Amount	Low	Fixed
DRE-11	Pausing Rewards Distributor Can Freeze Vault Entry and Exit	Informational	Fixed
DRE-12	Express Filler Debt Can Be Reimbursed to Incorrect Filler	Low	Acknowledged
DRE-13	Zero `stuckFundsRecipient` Breaks Composer Refund Fallback	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

- `contracts/*`

Repo: `https://github.com/dre-labs/dreusd`

Included Paths: `contracts/`

Extensions: `sol`

Files Excluded

- `contracts/mocks/*`

Operational Considerations

- **aUSDC Vault Must be a Secure Account**: The vault account holding protocol aUSDC is assumed to be a secure account, preferably behind a multisig. Misuse or compromise of vault keys can result in the complete loss of protocol funds.
- **Sufficient aUSDC Vault Approval**: Upon setting the vault address in the `dreAaveAdapter` contract, the vault is checked to have provided a non-zero allowance for aUSDC. However, if the allowance is not `type(uint256).max`, the cumulative transfers from the vault to the `dreAaveAdapter` may exhaust the allowance. Similarly, the vault may revoke the allowance, resetting it to zero at any time. The vault must maintain a sufficient aUSDC allowance for the `dreAaveAdapter` contract to ensure successful execution.
- **NFT Contract Receivers Must Implement** `onERC721Received()`: The `dreWithdrawalNFT` contract utilizes `_safeMint()` for NFT minting. Upon minting, the `dreWithdrawalNFT` contract expects the receiver to be either an EOA or a contract that implements `onERC721Received()`. Failure to meet either requirement will result in a failed mint.
- **Proper Configuration of** `dreRewardsDistributor`: It is critical that the `dreRewardsDistributor` contract properly configures the vault address as the `dreUSDs` contract. Improper configuration will result in the `dreRewardsDistributor` transferring dreUSD rewards to the wrong address while the `dreUSDs` contract will assume it holds the rewards. Eventually, dreUSDs shareholders will not be able to redeem their shares for dreUSD since the `dreUSDs` contract will have an insufficient dreUSD balance despite including the claimed rewards in its total assets calculation.
- **Withdrawals are Dependent on Privileged Executors**: Both standard and express withdrawals are not fulfilled until executed by privileged accounts. There is no guarantee that a pending withdrawal will be processed or that the custodian has sufficient USDC to fulfill withdrawal requests.
- **dreUSD Withdrawals are Always Filled with USDC**: Regardless of the underlying asset used to mint dreUSD initially, all dreUSD withdrawals are filled with USDC.
- **Reliance on External Protocols to Hold dreUSD Underlying Assets**: The DRE protocol moves at least a portion of user-deposited underlying assets into AAVE for yield generation. In return, the AAVE aUSDC tokens are held in a custodian vault, which can later be withdrawn from to fulfill dreUSD withdrawal requests. However, if AAVE is compromised, the underlying assets of dreUSD may be lost, potentially devaluing the token and preventing users from redeeming dreUSD for USDC.
- **Synchronization of Privileged State**: Administrators are responsible for ensuring that state that they control is appropriately synchronized across chains; for example, if it is desired to have all addresses frozen for dreUSD on Base also frozen for dreUSD on Ethereum, then the `GUARDIAN_ROLE` address must call the appropriate functions on the `dreUSD` contract on each chain.

Key Actors And Their Capabilities

`dreTimelockController` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE`: Configure and manage privileged roles.
- `PROPOSER_ROLE`: Schedule operations for future execution.
- `EXECUTOR_ROLE`: Execute pending operations.
- `CANCELLER_ROLE`: Cancel pending operations.

Trust Assumption

- `DEFAULT_ADMIN_ROLE`: Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `PROPOSER_ROLE`: Trusted to propose operations that will not affect the protocol negatively.
- `EXECUTOR_ROLE`: Trusted to execute legitimate pending operations promptly following the set delay.
- `CANCELLER_ROLE`: Trusted to cancel erroneous or unnecessary pending operations, preventing future execution.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
  - `grantRole()`: Grant privileged roles to trusted accounts.
  - `revokeRole()`: Revoke privileged roles from previously granted accounts.
  - `renounceRole()`: Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `PROPOSER_ROLE`
  - `schedule()` and `scheduleBatch()`: Schedule one or more operations for future execution.
- `EXECUTOR_ROLE`
  - `execute()` and `executeBatch()`: Execute pending scheduled operations following the configured delay.

- `CANCELLER_ROLE`
  - `cancel()` : Cancel a pending scheduled operation.

LayerZero Relayer

Responsibility

Relay cross-chain messages submitted to LayerZero from the OApp contracts (`dreShareOFT` , `dreShareOFTAdapter` , and `dreUSD`).

Trust Assumption

The relayer is trusted to frequently monitor the LayerZero endpoint contracts on all supported chains, listening for events indicating a cross-chain message has been sent. Cross-chain messages are expected to be relayed to the correct destination chain with the correct payload in a timely manner.

`dreShareOFT` Contract Owner

Responsibility

Configure and manage the contract sanctions list, account-freeze status, contract-upgrade authorization, and LayerZero settings.

Trust Assumption

Trusted to: properly set the sanctions list contract as the Chainalysis Sanctions Oracle; only freeze non-compliant accounts; promptly unfreeze compliant accounts; upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions; and configure LayerZero settings correctly, allowing users to successfully bridge tokens between supported chains.

Transferring this role to a malicious account can result in: compliant users being prevented from interacting with the protocol, including bridging and claiming underlying assets; implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data; and cross-chain message relaying to fail.

Exclusive Functions

- `setSanctionsList()` : Update the Chainalysis Sanctions Oracle contract address.
- `freeze()` : Freeze an account to prevent minting, burning, and transferring dreUSDs.
- `unfreeze()` : Unfreeze an account to allow minting, burning, and transferring dreUSDs.
- `upgradeToAndCall()` : Update the implementation contract address.
- `setMsgInspector()` : Set or update an optional contract used to inspect messages before they are relayed by LayerZero.
- `setPeer()` : Set or update the LayerZero peer address for a corresponding endpoint.
- `setDelegate()` : Set an optional delegate that can set configurations on behalf of this OApp directly in the LayerZero endpoint.
- `transferOwnership()` : Transfer ownership of the contract to another account.
- `renounceOwnership()` : Renounce the ownership role, leaving the contract without an owner.

`dreShareOFTAdapter` Contract Owner

Responsibility

Configure and manage the contract's LayerZero settings and authorize contract upgrades.

Trust Assumption

Trusted to: configure LayerZero settings correctly, allowing users to successfully bridge tokens between supported chains; upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions.

Transferring this role to a malicious account can result in: cross-chain message relaying failure; implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data.

Exclusive Functions

- `upgradeToAndCall()` : Update the implementation contract address.
- `setMsgInspector()` : Set or update an optional contract used to inspect messages before they are relayed by LayerZero.
- `setPeer()` : Set or update the LayerZero peer address for a corresponding endpoint.
- `setDelegate()` : Set an optional delegate that can set configurations on behalf of this OApp directly in the LayerZero endpoint.
- `transferOwnership()` : Transfer ownership of the contract to another account.
- `renounceOwnership()` : Renounce the ownership role, leaving the contract without an owner.

`dreAaveAdapter` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure the contract and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `WITHDRAWER_ROLE` : Withdraw USDC from AAVE using aUSDC held in the vault.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `WITHDRAWER_ROLE` : The withdrawer role should only be granted to the `dreUSDManager` contract to allow for the successful execution of the `dreUSDManager.fillWithdrawal()` function. **Any other account that is granted this role can use the entire balance of aUSDC held in the vault to claim USDC from AAVE.**

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`

    - `setVault()` : Update the vault address that holds protocol aUSDC.
    - `recoverToken()` : Withdraw any ERC20 tokens held by the contract and transfer them to a specified recipient.
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.
    - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
    - `upgradeToAndCall()` : Update the implementation contract address.
- `WITHDRAWER_ROLE`
    - `withdraw()` : Withdraw up to the full balance of aUSDC tokens held in the vault account to claim USDC from AAVE.

`dreRewardsDistributor` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `PAUSER_ROLE` : Manage the pause state for adding and claiming rewards.
- `MODERATOR_ROLE` : Add dreUSD rewards to the contract. As rewards vest and are claimed, they are transferred into the vault.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `PAUSER_ROLE` : Trusted to only pause the contract as needed and promptly unpause when acceptable. Granting this role to a malicious account can prevent users from accessing their funds in the `dreUSDs` vault.
- `MODERATOR_ROLE` : The moderator is trusted not to intentionally reset the vesting window to lower the distribution rate.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.
    - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
    - `upgradeToAndCall()` : Update the implementation contract address.
- `PAUSER_ROLE`
    - `pause()` : Paused the contract, restricting the execution of adding rewards and claiming vested rewards.
    - `unpause()` : Unpaused the contract, allowing the execution of adding rewards and claiming vested rewards.
- `MODERATOR_ROLE`
    - `addRewards()` : Add new dreUSD rewards to be distributed over the set vesting window.

`dreUSD` Contract Privileged Roles

Responsibility

- **Owner**: Manage LayerZero configurations.
- `DEFAULT_ADMIN_ROLE` : Configure the contract and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `MANAGER_ROLE` : Mint and burn dreUSD.
- `GUARDIAN_ROLE` : Manage the freezing and unfreezing of accounts, determining whether each can participate in protocol operations. Frozen accounts are prevented from:
    - Minting new dreUSD
    - Transferring previously minted dreUSD tokens, including bridging to other supported chains via LayerZero
    - Staking previously minted dreUSD in the `dreUSDs` contract
    - Transferring previously minted dreUSDs shares, including bridging to other supported chains via LayerZero
    - Unstaking from the `dreUSDs` contract

Trust Assumption

- **Owner**: Trusted to properly configure the LayerZero settings. Improper configurations can cause cross-chain message relaying to fail.
- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `MANAGER_ROLE` : Trusted to be granted only to the `dreUSDManager` contract. Granting this role to a malicious account can result in excessive unbacked dreUSD minting or the unrestricted burning of users' dreUSD.
- `GUARDIAN_ROLE` : Trusted to appropriately manage the freeze status of accounts. Granting this role to a malicious account can freeze honest accounts, preventing them from participating in protocol operations, such as minting, staking, and withdrawing previously deposited funds. Alternatively, a malicious account with this role can unfreeze sanctioned accounts, potentially rendering the protocol non-compliant.

Exclusive Functions

- **Owner**
    - `setMsgInspector()` : Set or update an optional contract used to inspect messages before they are relayed by LayerZero.
    - `setPeer()` : Set or update the LayerZero peer address for a corresponding endpoint.

    - `setDelegate()` : Set an optional delegate that can set configurations on behalf of this OApp directly in the LayerZero endpoint.
- `DEFAULT_ADMIN_ROLE`
    - `setSanctionsList()` : Set or update the sanctions list contract address.
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.
    - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
    - `transferOwnership()` : Transfers ownership of the contract to a new account.
    - `renounceOwnership()` : Renounce the current owner, leaving the contract without an owner. A new owner can be granted in the future using `transferOwnership()`.
- `UPGRADER_ROLE`
    - `upgradeToAndCall()` : Update the implementation contract address.
- `MANAGER_ROLE`
    - `mint()` : Mint dreUSD tokens to any account.
    - `burn()` : Burn dreUSD tokens from any account.
- `GUARDIAN_ROLE`
    - `freeze()` : Freeze an account, preventing them from participating in all protocol operations, including accessing previously deposited funds.
    - `unfreeze()` : Unfreeze an account, allowing them to participate in all protocol operations, including accessing previously deposited funds.

`dreUSDManager` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `KEEPER_ROLE` : Mint dreUSD for fiat deposits off-chain.
- `EXPRESS_OPERATOR_ROLE` : Fill pending express withdrawal requests.
- `TREASURY_ROLE` : Fill standard pending withdrawal requests and repay the express withdrawal filler.
- `MODERATOR_ROLE` : Configure and manage contract settings such as supported underlying assets, the vault address that holds deposited assets, custodians authorized to generate signatures for off-chain fiat deposits, and the maximum amount of fiat that can be used per day to mint dreUSD.
- `WITHDRAWAL_CONFIG_ROLE` : Configure and manage contract settings such as the express withdrawal fee, express withdrawal limit, and standard withdrawal delay.
- `PAUSER_ROLE` : Manage the pause state for dreUSD deposits and withdrawals.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `KEEPER_ROLE` : The keeper is trusted to mint dreUSD for fiat deposits off-chain in accordance to amounts verified by custodian signatures. A malicious keeper can delay minting dreUSD for users who deposited fiat off-chain.
- `EXPRESS_OPERATOR_ROLE` : Trusted to fill pending express withdrawal requests promptly.
- `TREASURY_ROLE` : Trusted to fill pending standard withdrawal requests promptly and repay the express withdrawal filler as necessary.
- `MODERATOR_ROLE` : Trusted to properly configure the contract settings. A malicious account with this role can: redirect deposited underlying assets into a compromised account; authorize malicious custodians who can generate signatures to mint dreUSD that is not backed by fiat deposits off-chain; remove supported underlying assets, preventing users from minting dreUSD.
- `WITHDRAWAL_CONFIG_ROLE` : Trusted to properly configure the contract withdrawal settings. A malicious account with this role can: Set the withdrawal fee to its maximum value (5%), forcing users to pay higher fees; set the withdrawal fee to 0%, preventing the protocol from claiming any fees; manipulate the standard withdraw delay.
- `PAUSER_ROLE` : Trusted to only pause the contract as needed and promptly unpause when acceptable. Granting this role to a malicious account can prevent users from minting dreUSD, initiating withdrawal requests, and fulfilling pending withdrawals, leaving users without dreUSD or the underlying USDC.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.
    - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
    - `upgradeToAndCall()` : Update the implementation contract address.
- `KEEPER_ROLE`
    - `mintFromUsd()` : Mint dreUSD to a recipient address using a valid custodian signature based on fiat deposited off-chain.
    - `mintRewards()` : Mint dreUSD to the `dreRewardsDistributor` using a valid custodian signature based on fiat deposited off-chain.
- `EXPRESS_OPERATOR_ROLE`
    - `fillExpressWithdrawals()` : Fill pending express withdrawals by transferring USDC from the caller to the withdrawal position NFT owner and express fee recipient.
- `TREASURY_ROLE`
    - `fillWithdrawal()` : Fill pending withdrawals with either USDC from the caller or USDC withdrawn from AAVE via the `dreAaveAdapter` contract.

    - `payExpressDebt()` : Repay the express withdrawal filler up to their current debt. Reduces the express filler's debt and increases the available amount for express withdrawals.
    - `adminWithdraw()` : Withdraw any ERC20 token held in the contract.
- `MODERATOR_ROLE`
    - `updateVault()` : Update the vault address that holds underlying deposited assets.
    - `updateCustodianList()` : Add or remove an authorized custodian who can generate valid signatures to mint dreUSD for off-chain fiat deposits.
    - `setDailyFiatMintCap()` : Update the maximum amount of fiat USD that can be used to mint dreUSD up to 100M USD.
    - `setDreRewardsDistributor()` : Update the `dreRewardsDistributor` contract address.
    - `updateAllowedList()` : Add or remove approved underlying tokens that can be used to mint dreUSD.
- `WITHDRAWAL_CONFIG_ROLE`
    - `updateExpressPaybackAddress()` : Update the account that is repaid for express withdrawals.
    - `updateExpressWithdrawal()` : Update the express withdrawal configuration. Set the express withdrawal fee up to 5%, the maximum express withdrawal limit, and the express fee recipient.
    - `updateWithdrawal()` Update the standard withdrawal waiting time to 1-14 days.
    - `updateVaultAdapter()` : Update the `dreAaveAdapter` contract address, which is optionally used to fill user withdrawal requests.
- `PAUSER_ROLE`
    - `pause()` : Pause the contract to prevent execution of dreUSD minting, withdrawal requests, and withdrawal filling.
    - `unpause()` : Unpause the contract to allow execution of dreUSD minting, withdrawal requests, and withdrawal filling.

`dreUSDOracle` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `MODERATOR_ROLE` : Manage oracle configurations.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `MODERATOR_ROLE` : The moderator is trusted to properly configure asset oracles with settings within reasonable bounds. Improper configurations or the removal of oracles for assets that remain active in the protocol can prevent users from minting dreUSD or initiating withdrawals. A malicious moderator can configure malicious oracles that report incorrect price data, allowing attackers to drain the protocol of underlying assets or causing honest users to receive less dreUSD than expected.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.
    - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE`, potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
    - `upgradeToAndCall()` : Update the implementation contract address.
- `MODERATOR_ROLE`
    - `setOracle()` : Add or update the Chainlink oracle for a specified token and set its staleness threshold.
    - `setStalenessThreshold()` : Update the staleness threshold for a previously set oracle within 1 minute to 24 hours.
    - `setDeviationThreshold()` : Update the deviation threshold for a previously set oracle up to 100%.
    - `removeOracle()` : Remove an oracle, resetting its staleness and deviation thresholds.
    - `setSequencerUptimeFeed()` : Update the sequencer uptime feed oracle address.
    - `setGracePeriod()` : Update the squencer uptime grace period.

`dreUSDs` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure the contract and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `PAUSER_ROLE` : Manage the pause state for deposit, withdrawal, and share transfer operations.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `PAUSER_ROLE` : Trusted to only pause the contract as needed and promptly unpause when acceptable. Granting this role to a malicious account can prevent users from accessing their funds in the vault.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
    - `setRewardsDistibutor()` : Set and update the `dreRewardsDistributor` contract address.
    - `grantRole()` : Grant privileged roles to trusted accounts.
    - `revokeRole()` : Revoke privileged roles from previously granted accounts.

- `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE` , potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
  - `upgradeToAndCall()` : Update the implementation contract address.
- `PAUSER_ROLE`
  - `pause()` : Pause the contract, preventing execution of depositing dreUSD for dreUSDs, redeeming dreUSDs for dreUSD, and transferring dreUSDs.
  - `unpause()` : Unpause the contract, allowing execution of depositing dreUSD for dreUSDs, redeeming dreUSDs for dreUSD, and transferring dreUSDs.

`dreWithdrawalNFT` Contract Privileged Roles

Responsibility

- `DEFAULT_ADMIN_ROLE` : Configure the contract and manage privileged roles.
- `UPGRADER_ROLE` : Authorize upgrades of the implementation contract.
- `MINTER_ROLE` : Mint NFTs and set their associated position data.
- `BURNER_ROLE` : Burn active NFTs and delete their associated position data.

Trust Assumption

- `DEFAULT_ADMIN_ROLE` : Users must trust that the default admin will only grant privileged roles to trusted accounts. Granting this role to a malicious account can result in the role being revoked from the original default admin and modifications to all other privileged roles.
- `UPGRADER_ROLE` : The upgrader is trusted to upgrade the implementation contract with properly audited code that functions as intended and does not cause storage collisions with previous versions. Granting this role to a malicious account can result in implementation contract upgrades that include malicious logic, brick the contract, or overwrite critical storage data, including other privileged roles.
- `MINTER_ROLE` : Trusted to be granted only to the `dreUSDManager` contract. Granting this role to a malicious account can result in unwanted NFT minting that represents positions with incorrect USDC amounts.
- `BURNER_ROLE` : Trusted to be granted only to the `dreUSDManager` contract. Granting this role to a malicious account can result in the burning of user NFTs and the deletion of their associated positions.

Exclusive Functions

- `DEFAULT_ADMIN_ROLE`
  - `setDreUSD()` : Update the dreUSD contract address used to determine whether an account is frozen or sanctioned.
  - `grantRole()` : Grant privileged roles to trusted accounts.
  - `revokeRole()` : Revoke privileged roles from previously granted accounts.
  - `renounceRole()` : Renounce the `DEFAULT_ADMIN_ROLE` , potentially leaving the contract without a default admin and preventing future management of other privileged roles.
- `UPGRADER_ROLE`
  - `upgradeToAndCall()` : Update the implementation contract address.
- `MINTER_ROLE`
  - `mint()` : Mint a new NFT and set its associated position data.
- `BURNER_ROLE`
  - `burn()` : Burns an existing NFT at the provided token ID and deletes the associated position.

Findings

DRE-1
dreUSD Manager Can Deflate Share Price During Reward Top-Up

Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `66f6d0630e3cfe270bb537ef1479e921c6bbcbc2` .

File(s) affected: `contracts/dreUSDManager.sol` , `contracts/dreRewardsDistributor.sol` , `contracts/dreUSDs.sol`

Description: `dreUSDManager.mintRewards()` mints dreUSD to the distributor and then calls `addRewards()` . Inside `addRewards()` , the distributor first calls `_claimVested()` , which transfers already vested rewards to the vault before processing new rewards. That transfer does not update `dreUSDs._virtualBalance` , so `totalAssets()` can drop even though tokens moved into vault custody. This means regular reward top-ups can create the same persistent accounting gap and share-price deflation as direct external `claimVested()` calls.

Recommendation: Ensure reward top-ups settle vested rewards through a vault-synced accounting path before schedule updates. A direct fix is to claim remaining vested rewards with vault balance sync before `addRewards()` , or make `addRewards()` report/sync claimed amounts so `_virtualBalance` is updated atomically.

DRE-2 `dreUSD` Ownership Can only Be Transferred Once

Medium Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `f964e37df7142d2e8b2e3263e84e23dcafe4d909` .

File(s) affected: `contracts/dreUSD.sol`

Description: The comment concerning `dreUSD.transferOwnership()` states that the intention of the overridden function is for calling to require the use of `DEFAULT_ADMIN_ROLE` *instead of* `owner`. However, due to the calling of `super.transferOwnership()`, the `onlyOwner` modifier is triggered, requiring the caller to possess both `DEFAULT_ADMIN_ROLE` and be `owner`.

Similarly, the comment concerning `dreUSD.renounceOwnership()` indicates that it is implemented to use `DEFAULT_ADMIN_ROLE` instead of `owner`, whereas the actual logic requires the caller to possess both attributes.

Hence, once ownership is transferred away from possessors of `DEFAULT_ADMIN_ROLE`, it will be impossible for ownership to be transferred again. Similarly, it will be impossible for ownership to be renounced. Ultimately, transferring ownership to an account that does not have `DEFAULT_ADMIN_ROLE` would fundamentally remove control of ownership from current possessors of `DEFAULT_ADMIN_ROLE`.

Exploit Scenario:
The following passing tests demonstrate the issue.

```
function test_TransferOwnership_RevertIf_NewOwnerTries() public {
    assertEq(token.owner(), defaultAdmin);
    address newOwner = makeAddr("newOwner");
    vm.prank(defaultAdmin);
    token.transferOwnership(newOwner);
    assertEq(token.owner(), newOwner);
    vm.prank(newOwner);
    vm.expectRevert(
        abi.encodeWithSelector(IAccessControl.AccessControlUnauthorizedAccount.selector, newOwner,
DEFAULT_ADMIN_ROLE)
    );
    token.transferOwnership(defaultAdmin);
    assertEq(token.owner(), newOwner);
}

function test_TransferOwnership_RevertIf_DefaultAdminTriesTransferTwice() public {
    assertEq(token.owner(), defaultAdmin);
    address newOwner = makeAddr("newOwner");
    vm.prank(defaultAdmin);
    token.transferOwnership(newOwner);
    assertEq(token.owner(), newOwner);
    vm.prank(defaultAdmin);
    vm.expectRevert(
        abi.encodeWithSelector(Ownable.OwnableUnauthorizedAccount.selector, defaultAdmin)
    );
    token.transferOwnership(defaultAdmin);
    assertEq(token.owner(), newOwner);
}
```

Recommendation: Modify `dreUSD.transferOwnership()` to perform the state modification directly within the function body instead of calling `super.transferOwnership()`.

DRE-3 Cross-Chain Messages Stuck in LayerZero Queue

• Low ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `2f4b406576a7cc3ff953d2be2a5832162d73e12f`.

File(s) affected: `contracts/ovault/dreShareAdapter.sol`

Description: dreUSDs bridged from an unfrozen account on a spoke chain to a frozen account on the hub chain will fail during the `dreShareOFTAdapter.lzReceive()` call. Unlike all other bridge flows, bridging dreUSDs to the hub chain transfers dreUSDs held in the `dreShareOFTAdapter` contract to the receiver. When a receiver is frozen or sanctioned, the transfer will fail. Therefore, the LayerZero message will fail and be trapped in the message queue.

Exploit Scenario:
1. Bob has 100 dreUSDs on a spoke chain
2. Bob calls `dreShareOFT.send()` to bridge his 100 dreUSDs to Alice, a frozen account on the hub chain
3. Bob's 100 dreUSDs is burnt on the spoke chain
4. LayerZero relays the cross-chain message to the hub chain
5. The following call trace occurs on the hub chain: `dreShareOFTAdapter.lzReceive()` → `_lzReceive()` → `_credit()` → `dreUSDs.safeTransfer(Alice, 100)` → `dreUSDs._update()` → `_validateAddress(Alice)`

6. `_validateAddress(Alice)` fails, causing the `dreShareOFTAdapter.lzReceive()` call to fail
7. Bob bridge message is stuck in the LayerZero message queue

Recommendation: Modify the `dreUSDs._update()` function to disregard the `_validateAddress()` checks when `from` is the `dreShareOFTAdapter` contract.

DRE-4 Excessive Access Controls for Sensitive Functionality

• Low ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `745de0d16e7bd38f0a6decd38e19253e4ffad4b1`.

File(s) affected: `contracts/dreAaveAdapter.sol`, `contracts/dreUSD.sol`, `contracts/dreWithdrawalNFT.sol`

Description: Many of the privileged roles in the protocol contracts can be granted to numerous contracts and accounts. For some functionality, this is acceptable as multiple accounts may be expected to maintain operational processes. However, some privileged functionality is expected to be accessible only to other contracts in the protocol. Execution of certain privileged functionality by EOAs could break accounting or easily drain protocol funds. As a result, managing access to these specific functions using the default behavior of `AccessControl`'s roles, which allows roles to be granted to multiple addresses, may be inappropriate.

The following privileged functions should have their access restricted to a single protocol contract:

1. `dreAaveAdapter.withdraw()` requires the caller to have the `WITHDRAWER_ROLE`. Only the `dreUSDManager` contract should have this role. Granting this role to any other account increases the risk of incorrect handling of protocol funds, as the caller can withdraw the full aUSDC balance from the vault account.
2. `dreUSD.mint()` and `dreUSD.burn()` requires the caller to have the `MANAGER_ROLE`. Only the `dreUSDManager` contract should have this role. Granting this role to any other account increases the risk of erroneous dreUSD minting and burning.
3. `dreWithdrawalNFT.mint()` requires the caller to have the `MINTER_ROLE`. Only the `dreUSDManager` contract should have this role. Granting this role to any other account increases the risk of erroneously creating positions, potentially allowing an attacker to steal protocol assets.
4. `dreWithdrawalNFT.burn()` requires the caller to have the `BURNER_ROLE`. Only the `dreUSDManager` contract should have this role. Granting this role to any other account increases the risk of erroneously deleting valid positions, preventing users from fulfilling their withdrawals and claiming their assets.

Recommendation: In all cases, consider restricting access by setting the `dreUSDManager` contract address upon initialization, and replace the `onlyRole(..._ROLE)` modifiers with the check:

```
if (msg.sender != dreUSDManager) revert InvalidCaller();
```

DRE-5 Unclaimable Donated dreUSD

• Low ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `5da7f99a38eee4c333129431a5b108343109573c`.

File(s) affected: `contracts/dreUSDs.sol`

Description: The `dreUSDs` contract uses `_virtualBalance` to track explicit deposits and vested dreUSD rewards. Donating dreUSD to the `dreUSDs` contract will not inflate the share price and cannot be redeemed with issued shares. Therefore, any donated dreUSD is unclaimable as the `dreUSDs` contract does not implement a mechanism to withdraw the difference is dreUSD between `dreUSD.balanceOf(dreUSDs)` and `_virtualBalance`.

Exploit Scenario:

1. Assume the following:
   - `dreUSDs` contract virtual balance is 10,000 dreUSD
   - 0 dreUSD rewards are vested and unclaimed in the `dreRewardsDistributor` contract
   - 7,000 dreUSDs shares are issued
2. 200 dreUSD is donated to the `dreUSDs` contract (dreUSD balance of `dreUSDs` is 10,200)
3. All 7,000 dreUSDs shares are redeemed, claiming the virtual balance of 10,000 dreUSD
4. `dreUSDs` still holds 200 dreUSD, which is trapped in the contract

Recommendation: Implement a mechanism to withdraw excess dreUSD from the `dreUSDs` contract above the `_virtualBalance`.

DRE-6 Spoke Share Compliance Can Drift From Local `dreUSD` Policy

• Low ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `39b7a39c553f0f9be9e8cf016028dc3ef3c204fc`.

File(s) affected: `contracts/ovault/dreShareOFT.sol`, `contracts/dreUSD.sol`

Description: On spoke chains, `dreShareOFT` keeps its own `sanctionsList` and `frozen` state, while local `dreUSD` enforces a separate compliance state. Because these are configured and updated independently, policy can diverge on the same chain. This can create inconsistent blocking behavior between `dreUSD` and share transfers, increase operational error risk, and weaken predictable compliance enforcement across bridge flows.

Recommendation: Use a single compliance source per chain by delegating `dreShareOFT` validation to local `dreUSD.validateAddress(...)`, or enforce strict atomic sync of sanctions/freeze updates across both contracts.

DRE-7
Bridge Sends Can Credit Blocked Recipients on Destination

Low · Acknowledged

Update
Marked as "Acknowledged" by the client.
Addressed in: `ae8d98b7e28984817d93791dcd059dd58facb7fe`.
The client provided the following explanation:

> Updated the docsa

File(s) affected: `contracts/ovault/dreShareOFT.sol`, `contracts/dreUSD.sol`, `contracts/ovault/dreOVaultComposer.sol`

Description: Bridge routing uses two recipient fields depending on path. In standard sends, the **outer** `SendParam.to` is the destination recipient. In compose sends, the outer `SendParam.to` is the compose receiver (for example, the hub composer), while the final beneficiary is the **inner** `SendParam.to` encoded in `composeMsg` and decoded in `handleCompose`. These destination fields are not pre-validated for sanctions/freeze before cross-chain credit, and destination `_credit` mints by bypassing address validation. If compliance state is not synchronized across chains, a blocked user can still be credited through bridge/compose routing on a chain where the address is not blocked, defeating local blocking intent. If sanctions/freeze configuration is synchronized on the destination chain, destination-side transfer restrictions still apply after credit.

Recommendation: Choose one explicit policy and apply it consistently:
1. Strict block policy: add pre-send destination compliance validation using a synchronized compliance source, and revert when destination recipient is blocked.
2. Quarantine policy: keep current delivery behavior, but document that blocked recipients can still be credited and define handling procedures for quarantined balances.

DRE-8
Distributor Rotation without Settlement Can Understate Vault Assets

Low · Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `c71713f92743a9947a1eab09ddfad961be3f5058`.

File(s) affected: `contracts/dreUSDs.sol`, `contracts/dreRewardsDistributor.sol`

Description: `dreUSDs.setRewardsDistributor()` changes the `rewardsDistributor` address used by the vault. Since `totalAssets()` reads `vestedAmount()` only from the currently configured distributor, changing this pointer without first claiming remaining rewards from the old distributor can drop those old rewards from accounting. If rewards are still vesting or unclaimed in the previous distributor, they remain excluded until separately settled, which can understate vault assets and share pricing during the transition.

Recommendation: Claim all remaining rewards and account them in the vault before switching `rewardsDistributor`.

DRE-9 `mintAndStake()` Lacks `minSharesOut` Protection

Low · Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `d0334fb704aad613b44476ec13909165695705af`.

File(s) affected: `contracts/dreUSDManager.sol`, `contracts/interfaces/IdreUSDManager.sol`

Description: `dreUSDManager.mintAndStake()` enforces `minAmountOut` only for the intermediate `dreUSD` mint, then deposits into `dreUSDs` without a minimum on final shares received. Because share conversion uses live `totalAssets()`, share output can move between quote and execution (for example when vested rewards change), and the transaction can still succeed with materially fewer shares than expected.

Recommendation: Add share-side slippage protection to the `mintAndStake()` path.
1. Extend the entrypoint with a `minSharesOut` parameter and revert if returned shares are below the user bound.
2. Keep `minAmountOut` for the mint step, but document that both checks are required for full end-to-end slippage protection.

DRE-10
Aave Adapter Withdrawals Are Not Bound to Requested Amount

Low | Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `dfa9f672db1aae6e2bebe29d8f92d1a5e8c19a8b`.

File(s) affected: `contracts/dreAaveAdapter.sol`, `contracts/dreUSDManager.sol`

Description: `dreUSDManager.fillWithdrawal()` passes a queued `usdcAmount` into `withdraw()`, but `dreAaveAdapter.withdraw()` calls Aave with adapter-wide `actualBalance` instead of requested `amount`. If residual or donated `aUSDC` exists on the adapter, a single fill can withdraw more than the queued position amount or fail unexpectedly due to liquidity constraints.

Recommendation: Make withdrawal execution strictly amount-bound. Use requested `amount` (or exact transfer delta) in the Aave withdraw call, and add an admin sweep path for residual adapter balances so stale `aUSDC` /USDC cannot affect user fills.

DRE-11
Pausing Rewards Distributor Can Freeze Vault Entry and Exit

Informational | Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `bed788e94dd80b1442cd4dde397649110c59616e`.

File(s) affected: `contracts/dreUSDs.sol`, `contracts/dreRewardsDistributor.sol`

Description: `dreUSDs` always calls `_claimVestedRewards()` in `_deposit` and `_withdraw`, and that path calls `IdreRewardsDistributor(rewardsDistributor).claimVested()`, which is gated by `whenNotPaused`. As a result, pausing the rewards distributor also causes vault deposits and withdrawals to revert, coupling reward pause state to core vault entry and exit.

Recommendation: Choose one model and enforce it consistently:
1. Decouple: make reward claiming best-effort in vault flows (skip claim when distributor is paused) while preserving accounting invariants.
2. Intentional coupling: keep current behavior, but clearly document that pausing rewards also pauses vault entry/exit.

DRE-12
Express Filler Debt Can Be Reimbursed to Incorrect Filler

Low | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> with regards to this, we intend to :
>
> 1. have only 1 provider but we added roles because this provider can use multiple wallets to fill express
> 2. get rid off this type of express withdrawal as we intend to have enough defi liquidity for the users to use that instead of queue withdrawal
>
> in case we want to integrate a secondary partner for this purpose we will change the expressPaybackAddress instead of his wallet directly to our multisig and we split funds manually accordingly through some governance flows. we therefor ack this issue as we do

not want to complicate or risk to introduce more attack vectors on a flow that we do not intend to keep.

File(s) affected: `contracts/dreUSDManager.sol`

Description: `fillExpressWithdrawals()` records express reimbursement in a global `expressFillerDebt` bucket, but it does not record which `EXPRESS_OPERATOR_ROLE` holder advanced the USDC for each fill. Later, `payExpressDebt()` always repays the global `expressPaybackAddress`. Because reimbursement is not attributed per filler, it is possible that fillers are not reimbursed whenever multiple express operators are used or when `expressPaybackAddress` is changed before outstanding debt is settled. The debt accounting also assumes the filler always advances both the user amount and the fee amount, but if `expressFeeRecipient == msg.sender`, the fee leg is effectively a self-transfer and should not increase reimbursable debt.

Recommendation: We recommend the following:

1. Track express debt per payee at fill time rather than in one global bucket.
2. If not incorporating (1), at least block `expressPaybackAddress` changes while `expressFillerDebt > 0`.
3. Prevent overlap between the filler/payback path and `expressFeeRecipient`, or compute debt from the filler's net transferred amount instead of `userAmount + feeAmount`.

DRE-13 Zero `stuckFundsRecipient` Breaks Composer Refund Fallback

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `742837828f3647d5e10010cccbe5dc2ab827d810`.

File(s) affected: `contracts/ovault/dreOVaultComposer.sol`

Description: `dreOVaultComposer.setStuckFundsRecipient()` allows `address(0)` and its NatSpec explicitly says zero may be used to "disable fallback" That mode is not actually implemented safely. When `VaultComposerSync.lzCompose()` catches a failed compose, it calls `_refund()`. If the normal OFT refund inside `_refund()` also fails, `dreOVaultComposer.refund()` attempts a final local fallback by transferring tokens to `stuckFundsRecipient`. With `stuckFundsRecipient == address(0)`, that token transfer reverts, so the fallback path fails precisely when it is needed. This breaks the contract's intended stuck-funds recovery behavior and can leave failed compose transfers unrecovered until operators repair configuration and retry.

Recommendation: If "disable fallback" is intended behavior, implement it explicitly with a dedicated branch and documented recovery procedure rather than letting the fallback path revert implicitly. Alternatively, reject `address(0)` in `setStuckFundsRecipient()` to preserve the invariant already enforced by the constructor.

Auditor Suggestions

S1 Ownership Can Be Renounced

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We acknowledge the observation regarding renounceOwnership() / renounceRole() and the potential loss of administrative control.
> In our case, this behavior is not intended to be used in practice, but we also do not plan to modify the current implementation at this stage.
> The main reasons are:
> Shared and battle-tested inheritance structure
> A number of our contracts rely on established inheritance trees from OpenZeppelin and related dependencies. We prefer not to introduce custom overrides across the codebase unless strictly necessary, in order to avoid unnecessary deviations from battle-tested logic.
> Inconsistent ownership patterns across modules
> Our system does not use a single uniform access-control model. Some contracts rely on different ownership/admin abstractions, and in some cases Ownable is only inherited transitively and its owner-gated functionality is not actually used. For example, 0layer follows its own dependency structure compared to the rest of the system. Applying a blanket override would therefore add complexity without providing proportional practical benefit.
> Operational assumption
> Administrative privileges are expected to be managed through controlled operational processes, and privileged roles are not intended to be renounced. In other words, this is a governance/operations constraint rather than a behavior we currently want to enforce in code everywhere.
> Given the above, we consider this a known and accepted risk, and we prefer to preserve the current architecture rather than introduce broad changes to established code solely to disable renounce flows.

Description: OpenZeppelin's `Ownable` and `OwnableUpgradeable` implement the `renounceOwnership()` function, which sets the `owner` variable to `address(0)`. Calling this function will cause any function guarded by the `onlyOwner` modifier to be unusable.

Similarly, OpenZeppelin's `AccessControl` and `AccessControlUpgradeable` implement the `renounceRole()` and `revokeRole()` functions, which by default enables the loss of all functionality controlled by `DEFAULT_ADMIN_ROLE`.

Hence, all contracts derived from the above are by default liable in certain cases to lose access to core administrative functionality.

Recommendation: Confirm that this is the intended behavior. If not, override and disable the `renounceOwnership()` function in contracts inheriting from `Ownable` or `OwnableUpgradeable`, and override the `_revokeRole()` function in contracts derived from `AccessControl` or `AccessControlUpgradeable` to prevent the loss of the last holder of `DEFAULT_ADMIN_ROLE`. For extra security, consider using a two-step process when transferring the ownership of the contract (e.g. `Ownable2Step` from OpenZeppelin).

S2 Unlocked Pragma

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `ce8c821796f55f59de1e0e89732a08a9a655946e`.

Related Issue(s): SWC-103

Description: Every Solidity file specifies in the header a version number of the format `pragma solidity (^)0.8.*`. The caret (`^`) before the version number implies an unlocked pragma, meaning that the compiler will use the specified version *and above*, hence the term "unlocked".

Recommendation: For consistency and to prevent unexpected behavior in the future, we recommend to remove the caret to lock the file onto a specific Solidity version.

S3 Use Namespaced Storage

Acknowledged

Update

Marked as "Acknowledged" by the client.
Addressed in: `79539e311b23bea5b2244e8c05495fa6c5ea51fe`.
The client provided the following explanation:

> Removed the comment on OpenZeppelin upgradeability convention

Description: Upgradeable contracts in the codebase are appending `uint256[50] private __gap;` to their storage layouts in order to make room for storage variables potentially introduced in future upgrades. Code comments mention the use of this in OpenZeppelin code. However, OpenZeppelin has transitioned to using namespaced storage, as defined in ERC-7201.

Recommendation: Consider using namespaced storage as a means of future-proofing the contracts' storage layouts.

S4 Missing Input Validation

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `813b3921cb45159a74e28b7ab92e67a9d721b5d8`.

Related Issue(s): SWC-123

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error. Consider the following instances:

- `dreUSD`
  - `setSanctionsList()`
    - does not revert when `_sanctionsList` is `sanctionsList`
  - `freeze()`
    - does not revert when `frozen[account]` is already true
  - `unfreeze()`
    - does not revert when `frozen[account]` is already false
- `dreUSDs.sol`
  - `setRewardDistributor()`
    - does not revert when `_rewardsDistributor` is `rewardsDistributor`
- `dreWithdrawalNFT.sol`
  - `setDreUSD()`
    - does not revert when `_dreUSD` is `dreUSD`
- `dreUSDOracle.sol`
  - `setOracle()`
    - does not revert when `oracleAddress` is `oracles[token]`
  - `setStalenessThreshold()`
    - does not revert when `stalenessThreshold` is `stalenessThresholds[token]`
  - `setDeviationThreshold()`
    - does not revert when `deviationBps` is `deviationThresholds[token]`
  - `setSequencerUptimeFeed()`
    - does not revert when `_sequencerUptimeFeed` is `sequencerUptimeFeed`
  - `setGracePeriod()`
    - does not revert when `_gracePeriod` is `gracePeriod`
    - does not validate the new grace period is within reasonable bounds
- `dreShareOFT.sol`
  - `setSanctionsList()`
    - does not revert when `_sanctionsList` is `sanctionsList`
  - `freeze()`
    - does not revert when `frozen[account]` is already true
  - `unfreeze()`
    - does not revert when `frozen[account]` is already false
- `dreUSDManager.sol`
  - `updateVault()`
    - does not revert when `_custodianVault` is `custodianVault`
  - `setDailyFiatMintCap()`
    - does not revert when `_cap` is `dailyFiatMintCapUsd`
  - `setDreRewardsDistributor()`
    - does not revert when `_dreRewardsDistributor` is `dreRewardsDistributor`
  - `updateExpressPaybackAddress()`
    - does not revert when `newAddress` is `expressPaybackAddress`
  - `updateExpressWithdrawal()`
    - does not revert when `maxLimit` is `expressWithdrawalMaxLimit`
    - does not revert when `feeBps` is `expressWithdrawalFeeBps`
    - does not revert when `feeRecipient` is `expressFeeRecipient`
  - `updateWithdrawal()`
    - does not revert when `waitingTime` is `withdrawalWaitingTime`
  - `updateVaultAdapter()`
    - does not revert when `adapter` is `withdrawalVaultAdapter`

Recommendation: We recommend adding the relevant checks.

S5 Application Monitoring Can Be Improved

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `8a764a8c1f4a438d576c287436c8516ee29fc598`.

File(s) affected: `contracts/dreWithdrawalNFT.sol`, `contracts/dreUSDManager.sol`

Description: The following functions update important state variables but do not emit events. Changes are not visible in logs and are harder to index or monitor.

1. `dreWithdrawalNFT.setDreUSD()`: Emit an event containing the updated `dreUSD` address
2. `dreUSDManager._checkAndUpdateDailyFiatMint()`: Emit an event containing the updated daily fiat minted amount

Recommendation: Consider emitting the events.

S6 Misaligned Code Comment

Fixed

Update
Marked as "Acknowledged" by the client.
Addressed in: `009cccfc95736bfa7b5a08b119a70387cc3baec0`.
The client provided the following explanation:

> Removed the comment

File(s) affected: `contracts/dreUSDManager.sol`

Description: The code comment in `fillExpressWithdrawals()` (L605) and `fillWithdrawals()` (L548) states that positions that do not exist should be skipped in the for-loop. This suggests that the remaining iterations of the for-loop should execute. However, if the position does not exist, an error is thrown, and the entire call fails.

Recommendation: Consider allowing the for-loop to `continue` if the position does not exist or update the code comment.

S7 Vault Adapter Updates Lack Asset Compatibility Validation

Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `53f8429115ff89be2fd4b3dc66c2a4a2ad0de029`.

File(s) affected: `contracts/dreUSDManager.sol`

Description: `updateVaultAdapter()` validates only non-zero address. Vault-backed fills then assume the configured adapter uses the same asset/unit semantics as manager `usdc` accounting. A misconfigured adapter can break this assumption and cause incorrect payout calculations or failed fills.

Recommendation: Validate adapter compatibility during `updateVaultAdapter()` (at minimum enforce adapter asset alignment with manager `usdc`) before accepting configuration.

S8 Reward Distributor Mismatch Can Break Accounting

Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `3cf18dd43b7fa6773110fbd44d10d6e253241509`.

File(s) affected: `contracts/dreUSDManager.sol`, `contracts/dreUSDs.sol`

Description: Reward mint routing and vault reward accounting are controlled by separate configurable addresses, and there is no on-chain consistency check between them. If `dreUSDManager.setDreRewardsDistributor()` and `dreUSDs.setRewardsDistributor()` diverge, rewards can accrue on one distributor while `totalAssets()` reads another, creating accounting drift and operational recovery burden.

Recommendation:

1. Add a consistency guard so distributor updates maintain manager/vault alignment, or provide a single source of truth for the active distributor.
2. Include an operational procedure step that validates both pointers before reward mints and after any config change.

S9 Use Named Constants for Scaling Values

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `44a7bf5af7c7a45bacbb85f739e37cdc695e948a`.

File(s) affected: `contracts/dreUSDOracle.sol`, `contracts/dreUSDManager.sol`

Description: The contracts use raw scaling numbers like `10_000` and `10 ** decimals` in many places. This makes the math harder to read and easier to change inconsistently.

1. `contracts/dreUSDOracle.sol:161` uses `10_000`.
2. `contracts/dreUSDOracle.sol:416` uses `10_000`.
3. `contracts/dreUSDOracle.sol:417` uses `10_000`.
4. `contracts/dreUSDManager.sol:222` uses `10_000_000e6`.
5. `contracts/dreUSDManager.sol:223` uses `10_000_000e6`.
6. `contracts/dreUSDManager.sol:674` uses `10_000`.
7. `contracts/dreUSDManager.sol:725` uses `10_000`.

Recommendation: Define shared constants/helpers for basis points and decimal scaling, then use them everywhere this math appears.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/dre-labs/dreusd`

- `0d2...347 ./contracts/dreAaveAdapter.sol`
- `5c9...36a ./contracts/dreRewardsDistributor.sol`
- `b4a...1b6 ./contracts/dreUSD.sol`
- `22d...aef ./contracts/dreUSDManager.sol`
- `736...c9b ./contracts/dreUSDOracle.sol`
- `78f...ffd ./contracts/dreUSDs.sol`
- `5d5...b04 ./contracts/dreWithdrawalNFT.sol`
- `628...a69 ./contracts/governance/dreTimelockController.sol`
- `9a1...2c0 ./contracts/interfaces/IAaveV3Adapter.sol`
- `dcf...80b ./contracts/interfaces/IAaveV3Pool.sol`
- `091...cfc ./contracts/interfaces/IDreUSDOracle.sol`

- `808...16a ./contracts/interfaces/ISanctionsList.sol`
- `977...e74 ./contracts/interfaces/IWithdrawalNFT.sol`
- `c50...429 ./contracts/interfaces/IdreRewardsDistributor.sol`
- `4f8...c69 ./contracts/interfaces/IdreUSD.sol`
- `5f9...2f8 ./contracts/interfaces/IdreUSDManager.sol`
- `ea3...9d4 ./contracts/interfaces/IdreUSDs.sol`
- `7b0...46f ./contracts/ovault/dreOVaultComposer.sol`
- `3f2...db8 ./contracts/ovault/dreShareOFT.sol`
- `43b...47d ./contracts/ovault/dreShareOFTAdapter.sol`

Test Suite Results

Test results were obtained using `forge test`.

Update: There are now 614 tests versus the previous 577.

```
Ran 1 test for test/DeployDreAaveAdapter.t.sol:DeployDreAaveAdapterTest
[PASS] test_DeployDreAaveAdapter() (gas: 3071522)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 157.56ms (50.33ms CPU time)

Ran 1 test for test/DeployDreRewardsDistributor.t.sol:DeployDreRewardsDistributorTest
[PASS] test_DeployDreRewardsDistributor() (gas: 1858899)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 157.69ms (50.68ms CPU time)

Ran 1 test for test/DeployWithdrawalNFT.t.sol:DeployWithdrawalNFTTest
[PASS] test_DeployWithdrawalNFT() (gas: 3355854)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 157.33ms (50.49ms CPU time)

Ran 1 test for test/DeployDreUSDManager.t.sol:DeployDreUSDManagerTest
[PASS] test_DeployDreUSDManager() (gas: 8642436)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 157.55ms (50.08ms CPU time)

Ran 1 test for test/DeployDreUSDs.t.sol:DeployDreUSDsTest
[PASS] test_DeployDreUSDs() (gas: 2822604)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 157.73ms (50.30ms CPU time)

Ran 2 tests for test/DeployDreUSDOracle.t.sol:DeployDreUSDOracleTest
[PASS] test_DeployDreUSDOracle() (gas: 2374656)
[PASS] test_DeployDreUSDOracle_BaseMainnet() (gas: 2114128)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.71ms (789.33µs CPU time)

Ran 10 tests for test/ovault/dreOVaultComposer.t.sol:dreOVaultComposerTest
[PASS] test_depositAndSend_Local_FallbackToMsgSender() (gas: 156940)
[PASS] test_depositAndSend_Local_RefundsNativeTokens() (gas: 124314)
[PASS] test_depositAndSend_Local_RefundsToRefundAddress() (gas: 142806)
[PASS] test_depositAndSend_Local_RevertsWhenBothReject() (gas: 591813)
[PASS] test_depositAndSend_Local_ZeroMsgValue() (gas: 110464)
[PASS] test_depositAndSend_MinAmountLD_NotZero() (gas: 639485)
[PASS] test_depositAndSend_PreservesMinAmountLD() (gas: 640407)
[PASS] test_depositAndSend_Vault_SlippageProtection() (gas: 88875)
[PASS] test_redeemAndSend_Local_RefundsNativeTokens() (gas: 158472)
[PASS] test_redeemAndSend_PreservesMinAmountLD() (gas: 615990)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 185.93ms (35.25ms CPU time)

Ran 35 tests for test/ovault/dreShareOFT.t.sol:dreShareOFTTest
[PASS] test_Burn() (gas: 3521247)
[PASS] test_Burn_RevertIf_FromFrozenAddress() (gas: 3541921)
[PASS] test_Burn_RevertIf_FromSanctionedAddress() (gas: 3571462)
[PASS] test_Credit_MintsToFrozenAddress_ThenTransferReverts() (gas: 3574293)
[PASS] test_Credit_MintsToSanctionedAddress_ThenTransferReverts() (gas: 3603853)
[PASS] test_Freeze() (gas: 46636)
[PASS] test_FreezeAndUnfreeze_MultipleTimes() (gas: 60204)
[PASS] test_Freeze_RevertIf_NotOwner() (gas: 17794)
[PASS] test_Freeze_RevertIf_ZeroAddress() (gas: 18213)
[PASS] test_Initialize() (gas: 29279)
```

```
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 19524)
[PASS] test_Initialize_RevertIf_DelegateIsZeroAddress() (gas: 3275211)
[PASS] test_Mint_AllowsMinting() (gas: 3514998)
[PASS] test_Mint_RevertIf_ToFrozenAddress() (gas: 3491616)
[PASS] test_Mint_RevertIf_ToSanctionedAddress() (gas: 3523133)
[PASS] test_SanctionsList_NotSet() (gas: 3550506)
[PASS] test_SanctionsList_ReturnsFalse() (gas: 3584727)
[PASS] test_SetSanctionsList() (gas: 47240)
[PASS] test_SetSanctionsList_CanBeZeroAddress() (gas: 23029)
[PASS] test_SetSanctionsList_RevertIf_NotOwner() (gas: 17817)
[PASS] test_Transfer() (gas: 3550904)
[PASS] test_TransferFrom() (gas: 3558367)
[PASS] test_TransferFrom_RevertIf_FromFrozenAddress() (gas: 3571891)
[PASS] test_TransferFrom_RevertIf_ToFrozenAddress() (gas: 3574365)
[PASS] test_Transfer_RevertIf_FromFrozenAddress() (gas: 3544552)
[PASS] test_Transfer_RevertIf_FromSanctionedAddress() (gas: 3574146)
[PASS] test_Transfer_RevertIf_ToFrozenAddress() (gas: 3546983)
[PASS] test_Transfer_RevertIf_ToSanctionedAddress() (gas: 3579595)
[PASS] test_Transfer_WorksAfterUnfreeze() (gas: 3559295)
[PASS] test_Unfreeze() (gas: 35959)
[PASS] test_Unfreeze_RevertIf_NotOwner() (gas: 17816)
[PASS] test_Unfreeze_RevertIf_ZeroAddress() (gas: 18205)
[PASS] test_Upgrade() (gas: 3225112)
[PASS] test_Upgrade_RevertIf_NotOwner() (gas: 3206707)
[PASS] test_ValidateAddress_SkipsZeroAddress() (gas: 3519598)
Suite result: ok. 35 passed; 0 failed; 0 skipped; finished in 31.86ms (29.73ms CPU time)

Ran 11 tests for test/ovault/dreShareOFTAdapter.t.sol:dreShareOFTAdapterTest
[PASS] test_Credit_ReturnsAmountReceivedLD() (gas: 2852123)
[PASS] test_Credit_RevertIf_InsufficientBalance() (gas: 2850445)
[PASS] test_Credit_SkipsValidationForZeroAddress() (gas: 2847966)
[PASS] test_Credit_TransfersToFrozenAddress() (gas: 2853318)
[PASS] test_Credit_TransfersToSanctionedAddress() (gas: 2853250)
[PASS] test_Credit_TransfersToValidAddress() (gas: 2856719)
[PASS] test_Initialize() (gas: 18332)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 17941)
[PASS] test_Initialize_RevertIf_DelegateIsZeroAddress() (gas: 2708638)
[PASS] test_Upgrade() (gas: 2650534)
[PASS] test_Upgrade_RevertIf_NotOwner() (gas: 2641609)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 1.60ms (965.96µs CPU time)

Ran 52 tests for test/dreUSDs.t.sol:dreUSDsTest
[PASS] test_ClaimRewards_RevertsIfClaimFails() (gas: 66975)
[PASS] test_Deposit() (gas: 161981)
[PASS] test_Deposit_AfterRewardsAccrued() (gas: 175804)
[PASS] test_Deposit_ClaimsRewards() (gas: 195283)
[PASS] test_Deposit_MultipleUsers() (gas: 214028)
[PASS] test_Deposit_RevertIf_Paused() (gas: 46472)
[PASS] test_Deposit_ToDifferentReceiver() (gas: 165454)
[PASS] test_Deposit_ZeroAmount() (gas: 69474)
[PASS] test_Initialize() (gas: 39303)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 20237)
[PASS] test_Initialize_RevertIf_AssetIsZeroAddress() (gas: 90545)
[PASS] test_Initialize_RevertIf_DefaultAdminIsZeroAddress() (gas: 90437)
[PASS] test_IsBlockedAddress_ReturnsTrue_WhenFrozen() (gas: 2422573)
[PASS] test_IsBlockedAddress_ReturnsTrue_WhenSanctioned() (gas: 2549420)
[PASS] test_MaxDeposit() (gas: 24764)
[PASS] test_MaxDeposit_MaxMint_ReturnZero_WhenReceiverBlocked() (gas: 56702)
[PASS] test_MaxDeposit_ReturnsZeroWhenPaused() (gas: 46484)
[PASS] test_MaxMint() (gas: 24795)
[PASS] test_MaxRedeem() (gas: 159372)
[PASS] test_MaxWithdraw() (gas: 167042)
[PASS] test_MaxWithdraw_MaxRedeem_ReturnZeroWhenPaused() (gas: 183707)
[PASS] test_MaxWithdraw_MaxRedeem_ReturnZero_WhenOwnerBlocked() (gas: 192245)
[PASS] test_Mint() (gas: 159245)
[PASS] test_Mint_ClaimsRewards() (gas: 195315)
[PASS] test_Mint_RevertIf_Paused() (gas: 46572)
[PASS] test_Pause() (gas: 44231)
[PASS] test_Pause_RevertIf_NotPauser() (gas: 18362)
[PASS] test_PreviewDeposit() (gas: 23312)
[PASS] test_PreviewMint() (gas: 23320)
[PASS] test_PreviewRedeem() (gas: 156422)
```

```
[PASS] test_PreviewWithdraw() (gas: 156433)
[PASS] test_Redeem() (gas: 178309)
[PASS] test_Redeem_ClaimsRewards() (gas: 217732)
[PASS] test_Redeem_RevertIf_Paused() (gas: 183787)
[PASS] test_Roles() (gas: 34958)
[PASS] test_SetRewardsDistributor() (gas: 353326)
[PASS] test_SetRewardsDistributor_RevertIf_NotAdmin() (gas: 339032)
[PASS] test_SetRewardsDistributor_RevertIf_ZeroAddress() (gas: 18402)
[PASS] test_TotalAssets_IncludesVestedRewards() (gas: 178081)
[PASS] test_TotalAssets_OnlyVaultBalance() (gas: 158107)
[PASS] test_TotalAssets_OnlyVestedRewards() (gas: 42957)
[PASS] test_TotalAssets_WithRewardsAfterDeposit() (gas: 178065)
[PASS] test_Unpause() (gas: 34153)
[PASS] test_Unpause_RevertIf_NotPauser() (gas: 47874)
[PASS] test_Upgrade() (gas: 2154059)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 2130029)
[PASS] test_Withdraw() (gas: 183773)
[PASS] test_Withdraw_AllShares() (gas: 137279)
[PASS] test_Withdraw_ClaimsRewards() (gas: 218188)
[PASS] test_Withdraw_RevertIf_Paused() (gas: 183833)
[PASS] test_Withdraw_WithAllowance() (gas: 198936)
[PASS] test_Withdraw_ZeroAmount() (gas: 177689)
Suite result: ok. 52 passed; 0 failed; 0 skipped; finished in 8.86ms (8.11ms CPU time)

Ran 56 tests for test/dreUSD.t.sol:dreUSDTest
[PASS] test_Burn() (gas: 41895)
[PASS] test_BurnFromZeroAddress() (gas: 18557)
[PASS] test_Burn_RevertIf_FromFrozenAddress() (gas: 51120)
[PASS] test_Burn_RevertIf_FromSanctionedAddress() (gas: 82686)
[PASS] test_Burn_RevertIf_NotManager() (gas: 22236)
[PASS] test_Credit_MintsToFrozenAddress_Quarantined() (gas: 81723)
[PASS] test_Credit_MintsToSanctionedAddress_Quarantined() (gas: 113266)
[PASS] test_Credit_MintsToValidAddress() (gas: 46780)
[PASS] test_Credit_ReturnsAmountReceivedLD() (gas: 42370)
[PASS] test_Credit_ToZeroAddress_MintsToDead() (gas: 41926)
[PASS] test_Credit_ZeroAmount_NoMint() (gas: 25677)
[PASS] test_Freeze() (gas: 46752)
[PASS] test_FreezeAndUnfreeze_MultipleTimes() (gas: 60857)
[PASS] test_Freeze_RevertIf_NotGuardian() (gas: 18049)
[PASS] test_Freeze_RevertIf_ZeroAddress() (gas: 18351)
[PASS] test_GrantManagerRole() (gas: 156143)
[PASS] test_Initialize() (gas: 47338)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 22567)
[PASS] test_Initialize_RevertIf_DefaultAdminIsZeroAddress() (gas: 92846)
[PASS] test_Initialize_RevertIf_GuardianIsZeroAddress() (gas: 92900)
[PASS] test_Initialize_RevertIf_UpgraderIsZeroAddress() (gas: 92814)
[PASS] test_Mint() (gas: 57308)
[PASS] test_MintToZeroAddress() (gas: 18603)
[PASS] test_Mint_RevertIf_NotManager() (gas: 20283)
[PASS] test_Mint_RevertIf_ToFrozenAddress() (gas: 51151)
[PASS] test_Mint_RevertIf_ToSanctionedAddress() (gas: 82846)
[PASS] test_Permit() (gas: 83961)
[PASS] test_RenounceOwnership_RevertIf_NotDefaultAdmin() (gas: 29095)
[PASS] test_RenounceOwnership_Succeeds_WhenDefaultAdmin() (gas: 23105)
[PASS] test_RevokeManagerRole() (gas: 31561)
[PASS] test_Roles() (gas: 32830)
[PASS] test_SanctionsList_CanBeZeroAddress() (gas: 23209)
[PASS] test_SanctionsList_NotSet() (gas: 57572)
[PASS] test_SanctionsList_ReturnsFalse() (gas: 95879)
[PASS] test_SanctionsList_UpdatedAfterFreeze() (gas: 106806)
[PASS] test_SetSanctionsList() (gas: 147454)
[PASS] test_SetSanctionsList_RevertIf_NotAdmin() (gas: 118274)
[PASS] test_Transfer() (gas: 57836)
[PASS] test_TransferFrom() (gas: 65544)
[PASS] test_TransferFrom_RevertIf_FromFrozenAddress() (gas: 76272)
[PASS] test_TransferFrom_RevertIf_ToFrozenAddress() (gas: 80638)
[PASS] test_TransferOwnership_RevertIf_NotDefaultAdmin() (gas: 30459)
[PASS] test_TransferOwnership_Succeeds_WhenDefaultAdmin() (gas: 29430)
[PASS] test_Transfer_RevertIf_FromFrozenAddress() (gas: 48802)
[PASS] test_Transfer_RevertIf_FromSanctionedAddress() (gas: 80387)
[PASS] test_Transfer_RevertIf_ToFrozenAddress() (gas: 53155)
[PASS] test_Transfer_RevertIf_ToSanctionedAddress() (gas: 85858)
```

```
[PASS] test_Unfreeze() (gas: 36189)
[PASS] test_Unfreeze_RevertIf_NotGuardian() (gas: 18048)
[PASS] test_Unfreeze_RevertIf_ZeroAddress() (gas: 18420)
[PASS] test_Upgrade() (gas: 4004340)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 3985925)
[PASS] test_ValidateAddress_RevertIf_FrozenAddress() (gas: 45896)
[PASS] test_ValidateAddress_RevertIf_FrozenAndSanctionedAddress_FrozenTakesPrecedence() (gas: 104011)
[PASS] test_ValidateAddress_RevertIf_SanctionedAddress() (gas: 77539)
[PASS] test_ValidateAddress_Succeeds_ForNormalAddress() (gas: 16982)
Suite result: ok. 56 passed; 0 failed; 0 skipped; finished in 101.48ms (96.44ms CPU time)

Ran 72 tests for test/dreWithdrawalNFT.t.sol:DreWithdrawalNFTTest
[PASS] test_Approve_Success() (gas: 262318)
[PASS] test_Burn_MultipleTokens() (gas: 547948)
[PASS] test_Burn_RevertIf_AlreadyBurned() (gas: 216133)
[PASS] test_Burn_RevertIf_NoBurnerRole() (gas: 237963)
[PASS] test_Burn_RevertIf_PositionNotFound() (gas: 20879)
[PASS] test_Burn_Success() (gas: 226717)
[PASS] test_Burn_WithCustomBurner() (gas: 242424)
[PASS] test_FullLifecycle() (gas: 274786)
[PASS] test_GetOriginalUser_AfterTransfer() (gas: 260886)
[PASS] test_GetOriginalUser_RevertIf_PositionNotFound() (gas: 16095)
[PASS] test_GetOriginalUser_Success() (gas: 233560)
[PASS] test_GetPendingRange_NoMints() (gas: 14999)
[PASS] test_GetPendingRange_OutOfOrderBurns() (gas: 550558)
[PASS] test_GetPendingRange_SequentialMintsAndBurns() (gas: 548612)
[PASS] test_GetPosition_RevertIf_PositionNotFound() (gas: 17513)
[PASS] test_GetPosition_Success() (gas: 234647)
[PASS] test_GetPositions_EmptyArray() (gas: 11760)
[PASS] test_GetPositions_RevertIf_PositionNotFound() (gas: 240673)
[PASS] test_GetPositions_SingleId() (gas: 236576)
[PASS] test_GetPositions_Success() (gas: 629524)
[PASS] test_GetTokensByIndexes_EmptyArray() (gas: 11802)
[PASS] test_GetTokensByIndexes_NonSequentialIndexes() (gas: 624208)
[PASS] test_GetTokensByIndexes_RevertIf_IndexOutOfBounds() (gas: 236524)
[PASS] test_GetTokensByIndexes_Success() (gas: 625484)
[PASS] test_GetUsdcAmount_RevertIf_PositionNotFound() (gas: 15970)
[PASS] test_GetUsdcAmount_Success() (gas: 233270)
[PASS] test_GrantRole() (gas: 82143)
[PASS] test_GrantRole_RevertIf_NotAdmin() (gas: 22891)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 730747)
[PASS] test_Initialize_RevertIf_ZeroDefaultAdmin() (gas: 3178468)
[PASS] test_Initialize_RevertIf_ZeroDreUSD() (gas: 2471605)
[PASS] test_Initialize_Success() (gas: 37712)
[PASS] test_Mint_MultipleTokens() (gas: 624205)
[PASS] test_Mint_RevertIf_NoMinterRole() (gas: 20604)
[PASS] test_Mint_RevertIf_ZeroAddress() (gas: 18475)
[PASS] test_Mint_RevertIf_ZeroAmount() (gas: 20655)
[PASS] test_Mint_Success() (gas: 241850)
[PASS] test_Mint_WithCustomMinter() (gas: 264680)
[PASS] test_MultipleUsers_MultiplePositions() (gas: 1010021)
[PASS] test_PositionExists_False() (gas: 12911)
[PASS] test_PositionExists_FalseAfterBurn() (gas: 214956)
[PASS] test_PositionExists_True() (gas: 233054)
[PASS] test_RevokeRole() (gas: 32264)
[PASS] test_Roles_InitialSetup() (gas: 40889)
[PASS] test_SafeTransferFrom_Success() (gas: 261681)
[PASS] test_SafeTransferFrom_WithData_Success() (gas: 262066)
[PASS] test_SetApprovalForAll_Success() (gas: 498671)
[PASS] test_SetDreUSD_RevertIf_NotDefaultAdmin() (gas: 29881)
[PASS] test_SetDreUSD_RevertIf_ZeroAddress() (gas: 28522)
[PASS] test_SetDreUSD_Success() (gas: 29560)
[PASS] test_SupportsInterface() (gas: 15868)
[PASS] test_TokenByIndex() (gas: 624671)
[PASS] test_TokenId_IncrementsCorrectly() (gas: 524688)
[PASS] test_TokenOfOwnerByIndex() (gas: 625928)
[PASS] test_TotalSupply() (gas: 377424)
[PASS] test_Transfer_Success() (gas: 265485)
[PASS] test_Update_RevertIf_ApprovedAddressIsSanctioned() (gas: 292970)
[PASS] test_Update_RevertIf_BurnWhenOwnerSanctioned() (gas: 286073)
[PASS] test_Update_RevertIf_FrozenAddress() (gas: 262830)
[PASS] test_Update_RevertIf_MintToSanctionedAddress() (gas: 131458)
```

```
[PASS] test_Update_RevertIf_SafeTransferFromToSanctioned() (gas: 266327)
[PASS] test_Update_RevertIf_TransferFromFrozenAddress() (gas: 286860)
[PASS] test_Update_RevertIf_TransferFromSanctionedAddress() (gas: 288207)
[PASS] test_Update_RevertIf_TransferToSanctionedAddress() (gas: 266257)
[PASS] test_Update_Success_ApproveSanctionedOperator() (gas: 292968)
[PASS] test_Update_Success_Burn() (gas: 215351)
[PASS] test_Update_Success_NormalTransfer() (gas: 258832)
[PASS] test_Update_Success_SafeTransferFrom() (gas: 261724)
[PASS] test_Update_Success_SetApprovalForAllSanctionedOperator() (gas: 292385)
[PASS] test_Update_Success_TransferAfterSanctionRemoved() (gas: 270946)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 2401616)
[PASS] test_Upgrade_Success() (gas: 2412321)
Suite result: ok. 72 passed; 0 failed; 0 skipped; finished in 7.19ms (6.20ms CPU time)

Ran 84 tests for test/dreUSDOracle.t.sol:dreUSDOracleTest
[PASS] testFuzz_ValidatePrice_MatchesSpec(uint256,uint256,bool) (runs: 256, μ: 115100, ~: 115361)
[PASS] test_GetLatestPrice_RevertIf_OracleNotSet() (gas: 18243)
[PASS] test_GetLatestPrice_Works() (gas: 112112)
[PASS] test_GetPriceDecimals_AfterSetOracle() (gas: 104886)
[PASS] test_GetPriceDecimals_Works() (gas: 18318)
[PASS] test_GetTokenAmount_Dai18Decimals_Price18Decimals() (gas: 389361)
[PASS] test_GetTokenAmount_Dai18Decimals_Price8Decimals() (gas: 128574)
[PASS] test_GetTokenAmount_RevertIf_GracePeriodNotOver() (gas: 396633)
[PASS] test_GetTokenAmount_RevertIf_OracleNotSet() (gas: 18378)
[PASS] test_GetTokenAmount_RevertIf_PriceAboveDeviation() (gas: 131950)
[PASS] test_GetTokenAmount_RevertIf_PriceBelowDeviation() (gas: 131935)
[PASS] test_GetTokenAmount_RevertIf_PriceNonPositive() (gas: 118946)
[PASS] test_GetTokenAmount_RevertIf_SequencerDown() (gas: 410661)
[PASS] test_GetTokenAmount_RevertIf_Stale() (gas: 124375)
[PASS] test_GetTokenAmount_USDC_GreaterThan1() (gas: 136052)
[PASS] test_GetTokenAmount_USDC_LowerThan1() (gas: 131381)
[PASS] test_GetTokenAmount_USDC_Price18Decimals_GreaterThan1() (gas: 394035)
[PASS] test_GetTokenAmount_USDC_Price18Decimals_LowerThan1() (gas: 392462)
[PASS] test_GetTokenAmount_Usdc6Decimals_Price18Decimals() (gas: 389406)
[PASS] test_GetTokenAmount_Usdc6Decimals_Price8Decimals() (gas: 128644)
[PASS] test_GetTokenAmount_WorksAfterGracePeriod() (gas: 407240)
[PASS] test_GetUsdValue_CallsCheckSequencerStatus_BeforePrice() (gas: 410678)
[PASS] test_GetUsdValue_Dai18Decimals_Price8Decimals() (gas: 127160)
[PASS] test_GetUsdValue_PassesIf_DeviationThresholdNotSet() (gas: 112757)
[PASS] test_GetUsdValue_PassesIf_PriceWithinDeviation() (gas: 134605)
[PASS] test_GetUsdValue_RevertIf_GracePeriodNotOver() (gas: 396655)
[PASS] test_GetUsdValue_RevertIf_OracleNotSet() (gas: 18356)
[PASS] test_GetUsdValue_RevertIf_PriceAboveDeviation() (gas: 131108)
[PASS] test_GetUsdValue_RevertIf_PriceBelowDeviation() (gas: 131072)
[PASS] test_GetUsdValue_RevertIf_PriceNonPositive() (gas: 118102)
[PASS] test_GetUsdValue_RevertIf_SequencerDown() (gas: 410724)
[PASS] test_GetUsdValue_RevertIf_Stale() (gas: 123531)
[PASS] test_GetUsdValue_USDC_GreaterThan1() (gas: 134617)
[PASS] test_GetUsdValue_USDC_LowerThan1() (gas: 129965)
[PASS] test_GetUsdValue_Usdc6Decimals_Price8Decimals() (gas: 127187)
[PASS] test_GetUsdValue_WorksAfterGracePeriod() (gas: 405872)
[PASS] test_GetUsdValue_WorksWhenSequencerNotSet() (gas: 127176)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 24723)
[PASS] test_Initialize_RevertIf_DefaultAdminIsZeroAddress() (gas: 1969045)
[PASS] test_Initialize_RevertIf_ModeratorIsZeroAddress() (gas: 1969167)
[PASS] test_Initialize_RevertIf_SequencerUptimeFeedIsZeroAddress() (gas: 1969153)
[PASS] test_Initialize_RevertIf_UpgraderIsZeroAddress() (gas: 1969101)
[PASS] test_Initialize_SetsRoles() (gas: 26854)
[PASS] test_Initialize_SetsSequencerFeed() (gas: 18062)
[PASS] test_Initialize_SetsSequencerFeedWithAddress() (gas: 2114481)
[PASS] test_RemoveOracle_ClearsDeviationThreshold() (gas: 86068)
[PASS] test_RemoveOracle_RevertIf_NotSet() (gas: 23229)
[PASS] test_RemoveOracle_Works() (gas: 83680)
[PASS] test_SequencerCheck_HandlesStartedAtZero() (gas: 380392)
[PASS] test_SequencerCheck_WorksWithCustomGracePeriod() (gas: 412246)
[PASS] test_SetDeviationThreshold_RevertIf_ExceedsMax() (gas: 105403)
[PASS] test_SetDeviationThreshold_RevertIf_OracleNotSet() (gas: 23280)
[PASS] test_SetDeviationThreshold_Works() (gas: 108551)
[PASS] test_SetGracePeriod_EmitsEvent() (gas: 25906)
[PASS] test_SetGracePeriod_Works() (gas: 25450)
[PASS] test_SetOracle_AcceptsMinAndMaxStalenessThreshold() (gas: 191843)
[PASS] test_SetOracle_RevertIf_InvalidOracleDecimals() (gas: 294622)
```

```
[PASS] test_SetOracle_RevertIf_InvalidOracleInterface_DecimalsFails() (gas: 92062)
[PASS] test_SetOracle_RevertIf_InvalidOracleInterface_LatestRoundDataFails() (gas: 96267)
[PASS] test_SetOracle_RevertIf_InvalidOraclePrice_AboveThreshold() (gas: 38735)
[PASS] test_SetOracle_RevertIf_InvalidOraclePrice_BelowThreshold() (gas: 38744)
[PASS] test_SetOracle_RevertIf_InvalidOraclePrice_ZeroOrNegative() (gas: 33975)
[PASS] test_SetOracle_RevertIf_StalenessThresholdAboveMax() (gas: 23504)
[PASS] test_SetOracle_RevertIf_StalenessThresholdBelowMin() (gas: 23450)
[PASS] test_SetOracle_RevertIf_ZeroAddressesOrThreshold() (gas: 41536)
[PASS] test_SetOracle_Works() (gas: 107343)
[PASS] test_SetSequencerUptimeFeed_EmitsEvent() (gas: 322644)
[PASS] test_SetSequencerUptimeFeed_Works() (gas: 320053)
[PASS] test_SetStalenessThreshold_RevertIf_BelowMinOrAboveMax() (gas: 109219)
[PASS] test_SetStalenessThreshold_RevertIf_OracleNotSet() (gas: 23256)
[PASS] test_SetStalenessThreshold_RevertIf_ThresholdZero() (gas: 105395)
[PASS] test_SetStalenessThreshold_Works() (gas: 108551)
[PASS] test_Upgrade_RevertIf_ModeratorWithoutUpgraderRole() (gas: 1897758)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 1897789)
[PASS] test_Upgrade_Success() (gas: 1910400)
[PASS] test_ValidatePrice_FalseIf_GracePeriodNotOver() (gas: 396012)
[PASS] test_ValidatePrice_FalseIf_NoOracle() (gas: 15020)
[PASS] test_ValidatePrice_FalseIf_PriceExceedsDeviation() (gas: 128762)
[PASS] test_ValidatePrice_FalseIf_PriceNonPositive() (gas: 117177)
[PASS] test_ValidatePrice_FalseIf_SequencerDown() (gas: 410333)
[PASS] test_ValidatePrice_FalseIf_Stale() (gas: 122431)
[PASS] test_ValidatePrice_TrueAfterGracePeriod() (gas: 399960)
[PASS] test_ValidatePrice_TrueIf_PriceWithinDeviation() (gas: 128763)
[PASS] test_ValidatePrice_TrueIf_Valid() (gas: 121322)
Suite result: ok. 84 passed; 0 failed; 0 skipped; finished in 129.73ms (147.05ms CPU time)

Ran 148 tests for test/dreUSDManager.t.sol:dreUSDManagerTest
[PASS] testFuzz_ConvertToDecimals(uint256,uint256,uint256) (runs: 256, μ: 4537685, ~: 4537569)
[PASS] testMintAndStake_RevertIf_FrozenSender() (gas: 57421)
[PASS] testMintAndStake_RevertIf_SanctionedReceiver() (gas: 73364)
[PASS] testMintAndStake_RevertIf_SanctionedSender() (gas: 65337)
[PASS] testRemoveCustodian_RevertIf_NotAllowed() (gas: 22693)
[PASS] test_AddCustodian() (gas: 46427)
[PASS] test_AddCustodian_RevertIf_AlreadyAdded() (gas: 23456)
[PASS] test_AddCustodian_RevertIf_ZeroAddress() (gas: 18631)
[PASS] test_AddStablecoin() (gas: 482364)
[PASS] test_AddStablecoin_RevertIf_AlreadyAllowed() (gas: 23458)
[PASS] test_AddStablecoin_RevertIf_ZeroAddress() (gas: 18654)
[PASS] test_AdminWithdraw() (gas: 63029)
[PASS] test_AdminWithdraw_RevertIf_NotTreasury() (gas: 20474)
[PASS] test_AdminWithdraw_RevertIf_ZeroAddress() (gas: 30381)
[PASS] test_AdminWithdraw_RevertIf_ZeroAmount() (gas: 20956)
[PASS] test_CalculateExpressFee() (gas: 13161)
[PASS] test_Constructor_RevertIf_DreUSDZero() (gas: 53875)
[PASS] test_Constructor_RevertIf_DreUSDsZero() (gas: 53885)
[PASS] test_Constructor_RevertIf_ExpressWithdrawalNFTZero() (gas: 53932)
[PASS] test_Constructor_RevertIf_OracleZero() (gas: 53979)
[PASS] test_Constructor_RevertIf_UsdcZero() (gas: 53857)
[PASS] test_Constructor_RevertIf_WithdrawalNFTZero() (gas: 53986)
[PASS] test_ConvertToDecimals_FromEqualsTo() (gas: 675088)
[PASS] test_ConvertToDecimals_FromGreaterThanTo() (gas: 707805)
[PASS] test_ConvertToDecimals_FromLessThanTo() (gas: 169512)
[PASS] test_FillExpressWithdrawals() (gas: 452440)
[PASS] test_FillExpressWithdrawals_RevertIf_Paused() (gas: 487897)
[PASS] test_FillExpressWithdrawals_Reverts() (gas: 487252)
[PASS] test_FillWithdrawal() (gas: 361568)
[PASS] test_FillWithdrawal_RevertIf_Paused() (gas: 431651)
[PASS] test_FillWithdrawal_Reverts() (gas: 1017645)
[PASS] test_FillWithdrawal_WithVault() (gas: 388624)
[PASS] test_GetDailyFiatMinted() (gas: 174079)
[PASS] test_GetExpressAvailable() (gas: 12716)
[PASS] test_GetExpressFillerDebt() (gas: 12630)
[PASS] test_Initialize() (gas: 108812)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 41181)
[PASS] test_Initialize_RevertIf_DefaultAdminIsZeroAddress() (gas: 111562)
[PASS] test_Initialize_RevertIf_DreRewardsDistributorIsZeroAddress() (gas: 111639)
[PASS] test_Initialize_RevertIf_ExpressFeeRecipientIsZeroAddress() (gas: 111660)
[PASS] test_Initialize_RevertIf_ExpressPaybackAddressIsZeroAddress() (gas: 111671)
[PASS] test_MintAndStake() (gas: 289770)
```

```
[PASS] test_MintAndStake_RevertIf_OrderExpired() (gas: 25825)
[PASS] test_MintAndStake_RevertIf_Paused() (gas: 48386)
[PASS] test_MintAndStake_RevertIf_SlippageExceeded() (gas: 223835)
[PASS] test_MintAndStake_RevertIf_StablecoinNotAllowed() (gas: 461050)
[PASS] test_MintAndStake_RevertIf_ZeroAmount() (gas: 21072)
[PASS] test_MintAndStake_RevertIf_ZeroReceiver() (gas: 18910)
[PASS] test_MintFrom() (gas: 248761)
[PASS] test_MintFromUsd() (gas: 176785)
[PASS] test_MintFromUsd_RevertIf_CapIsZero() (gas: 44509)
[PASS] test_MintFromUsd_RevertIf_DailyFiatMintCapExceeded() (gas: 180141)
[PASS] test_MintFromUsd_RevertIf_InvalidChainId() (gas: 30150)
[PASS] test_MintFromUsd_RevertIf_InvalidSignature() (gas: 88939)
[PASS] test_MintFromUsd_RevertIf_MintExpired() (gas: 26468)
[PASS] test_MintFromUsd_RevertIf_MintRefAlreadyUsed() (gas: 177092)
[PASS] test_MintFromUsd_RevertIf_Paused() (gas: 83453)
[PASS] test_MintFromUsd_RevertIf_ReceiverIsDreRewardsDistributor() (gas: 59637)
[PASS] test_MintFromUsd_RevertIf_ZeroAmount() (gas: 25933)
[PASS] test_MintFromUsd_RevertIf_ZeroReceiver() (gas: 23904)
[PASS] test_MintFrom_RevertIf_InvalidAuthorizeSignature() (gas: 96584)
[PASS] test_MintFrom_RevertIf_OrderExpired() (gas: 28382)
[PASS] test_MintFrom_RevertIf_Paused() (gas: 53448)
[PASS] test_MintFrom_RevertIf_SlippageExceeded() (gas: 257676)
[PASS] test_MintFrom_RevertIf_StablecoinNotAllowed() (gas: 463619)
[PASS] test_MintFrom_RevertIf_ZeroAmount() (gas: 23630)
[PASS] test_MintFrom_RevertIf_ZeroFrom() (gas: 25916)
[PASS] test_MintFrom_RevertIf_ZeroReceiver() (gas: 25879)
[PASS] test_MintRewards() (gas: 229397)
[PASS] test_MintRewards_CallsAddRewards() (gas: 220406)
[PASS] test_MintRewards_RevertIf_InvalidCustodianSignature() (gas: 88962)
[PASS] test_MintRewards_RevertIf_InvalidReceiver() (gas: 58805)
[PASS] test_MintRewards_RevertIf_MintExpired() (gas: 30016)
[PASS] test_MintRewards_RevertIf_Paused() (gas: 82111)
[PASS] test_MintRewards_RevertIf_ZeroAmount() (gas: 29596)
[PASS] test_MintRewards_RevertIf_ZeroDistributor() (gas: 57233)
[PASS] test_Mint_AllFunctions_RevertIf_ZeroCustodianVault() (gas: 523492)
[PASS] test_Mint_RevertIf_CustodianNotSet() (gas: 613179)
[PASS] test_Mint_RevertIf_OrderExpired() (gas: 23182)
[PASS] test_Mint_RevertIf_Paused() (gas: 48406)
[PASS] test_Mint_RevertIf_SlippageExceeded() (gas: 164511)
[PASS] test_Mint_RevertIf_StablecoinNotAllowed() (gas: 460886)
[PASS] test_Mint_RevertIf_ZeroAmount() (gas: 18449)
[PASS] test_Mint_WithApproval() (gas: 171628)
[PASS] test_Mint_WithPermit() (gas: 210995)
[PASS] test_Mint_WithPermit_AllowanceAlreadyGiven_SkipsPermit() (gas: 186683)
[PASS] test_Mint_WithPermit_RevertIf_InvalidSignature() (gas: 109767)
[PASS] test_Mint_WithPermit_RevertIf_OrderExpired0() (gas: 23639)
[PASS] test_Mint_WithPermit_RevertIf_Paused() (gas: 48746)
[PASS] test_Mint_WithPermit_RevertIf_SlippageExceeded() (gas: 219478)
[PASS] test_Mint_WithPermit_RevertIf_StablecoinNotAllowed() (gas: 458702)
[PASS] test_Mint_WithPermit_RevertIf_ZeroAmount() (gas: 18768)
[PASS] test_PayExpressDebt() (gas: 528271)
[PASS] test_PayExpressDebt_AsTreasury() (gas: 519405)
[PASS] test_PayExpressDebt_LimitHeadroom() (gas: 552578)
[PASS] test_PayExpressDebt_RevertIf_ExceedsDebt() (gas: 414238)
[PASS] test_PayExpressDebt_RevertIf_NoPaybackAddressSet() (gas: 500372)
[PASS] test_PayExpressDebt_RevertIf_ZeroAmount() (gas: 18436)
[PASS] test_RemoveCustodian() (gas: 25445)
[PASS] test_RemoveStablecoin() (gas: 25337)
[PASS] test_RemoveStablecoin_RevertIf_NotAllowed() (gas: 458622)
[PASS] test_RequestExpressWithdrawal() (gas: 389244)
[PASS] test_RequestExpressWithdrawal_ExpressAmountExceedsAvailable() (gas: 522965)
[PASS] test_RequestExpressWithdrawal_RevertIf_FeeRecipientNotSet() (gas: 4613163)
[PASS] test_RequestExpressWithdrawal_RevertIf_NoExpressAvailable() (gas: 438735)
[PASS] test_RequestExpressWithdrawal_RevertIf_OrderExpired() (gas: 34376)
[PASS] test_RequestExpressWithdrawal_RevertIf_Paused() (gas: 46145)
[PASS] test_RequestExpressWithdrawal_RevertIf_SlippageExceeded() (gas: 206813)
[PASS] test_RequestExpressWithdrawal_RevertIf_SlippageExceeded_AtExpressLimit() (gas: 388601)
[PASS] test_RequestExpressWithdrawal_RevertIf_ZeroAmount() (gas: 31851)
[PASS] test_RequestWithdrawal() (gas: 350263)
[PASS] test_RequestWithdrawal_RevertIf_OrderExpired() (gas: 32339)
[PASS] test_RequestWithdrawal_RevertIf_Paused() (gas: 46209)
[PASS] test_RequestWithdrawal_RevertIf_Sanctioned() (gas: 58028)
```

```
[PASS] test_RequestWithdrawal_RevertIf_SlippageExceeded() (gas: 199608)
[PASS] test_RequestWithdrawal_RevertIf_ZeroAmount() (gas: 29402)
[PASS] test_SetDailyFiatMintCap() (gas: 42693)
[PASS] test_SetDreRewardsDistributor() (gas: 352750)
[PASS] test_SetDreRewardsDistributor_RevertIf_ZeroAddress() (gas: 18484)
[PASS] test_SupportsInterface_AccessControl() (gas: 10848)
[PASS] test_SupportsInterface_ERC165() (gas: 10876)
[PASS] test_SupportsInterface_Unsupported() (gas: 12406)
[PASS] test_Unpause_RevertIf_NotPauser() (gas: 18415)
[PASS] test_Unpause_Success() (gas: 34032)
[PASS] test_UpdateExpressPaybackAddress() (gas: 27684)
[PASS] test_UpdateExpressPaybackAddress_RevertIf_ZeroAddress() (gas: 18505)
[PASS] test_UpdateExpressWithdrawal_ClampsAvailableToMaxLimit() (gas: 43460)
[PASS] test_UpdateExpressWithdrawal_Fee() (gas: 49034)
[PASS] test_UpdateExpressWithdrawal_FeeRecipient() (gas: 48446)
[PASS] test_UpdateExpressWithdrawal_MaxFeeAllowed() (gas: 37653)
[PASS] test_UpdateExpressWithdrawal_MaxLimit() (gas: 49095)
[PASS] test_UpdateExpressWithdrawal_RevertIf_FeeOverMax() (gas: 20825)
[PASS] test_UpdateExpressWithdrawal_RevertIf_ZeroFeeRecipient() (gas: 18707)
[PASS] test_UpdateExpressWithdrawal_RevertIf_ZeroMaxLimit() (gas: 20730)
[PASS] test_UpdateVault() (gas: 27680)
[PASS] test_UpdateVaultAdapter() (gas: 300023)
[PASS] test_UpdateVaultAdapter_RevertIf_ZeroAddress() (gas: 18420)
[PASS] test_UpdateVault_RevertIf_ZeroAddress() (gas: 18486)
[PASS] test_UpdateWithdrawal_AcceptsBoundaries() (gas: 30761)
[PASS] test_UpdateWithdrawal_RevertIf_AboveMax() (gas: 18863)
[PASS] test_UpdateWithdrawal_RevertIf_BelowMin() (gas: 28454)
[PASS] test_UpdateWithdrawal_WaitingTime() (gas: 36747)
[PASS] test_Upgrade_RevertIf_DefaultAdminWithoutUpgraderRole() (gas: 4558447)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 4525516)
[PASS] test_Upgrade_Success() (gas: 4534973)
[PASS] test_setDailyFiatMintCap_AllowsMaxCap() (gas: 44025)
[PASS] test_setDailyFiatMintCap_AllowsZero() (gas: 22799)
[PASS] test_setDailyFiatMintCap_RevertIf_ExceedsMaxCap() (gas: 20158)
Suite result: ok. 148 passed; 0 failed; 0 skipped; finished in 134.20ms (172.57ms CPU time)

Ran 46 tests for test/dreAaveAdapter.t.sol:DreAaveAdapterTest
[PASS] testFuzz_GetAvailableBalance(uint256,uint256,uint256) (runs: 256, μ: 76940, ~: 77776)
[PASS] testFuzz_Withdraw(uint256) (runs: 256, μ: 120054, ~: 120863)
[PASS] test_GetAToken() (gas: 14833)
[PASS] test_GetAavePool() (gas: 14899)
[PASS] test_GetAvailableBalance() (gas: 31216)
[PASS] test_GetAvailableBalance_CombinedElseBranches() (gas: 68937)
[PASS] test_GetAvailableBalance_WhenAllowanceExceedsBalance() (gas: 55679)
[PASS] test_GetAvailableBalance_WhenLiquidityExceedsAvailable() (gas: 59867)
[PASS] test_GetAvailableBalance_WithLimitedPoolLiquidity() (gas: 46447)
[PASS] test_GetAvailableBalance_WithPartialAllowance() (gas: 44368)
[PASS] test_GetAvailableBalance_WithPartialVaultBalance() (gas: 46437)
[PASS] test_GetUsdc() (gas: 14802)
[PASS] test_GetVault() (gas: 14811)
[PASS] test_Initialize_RevertIf_ATokenAddressHasNoCode() (gas: 1887143)
[PASS] test_Initialize_RevertIf_ZeroATokenAddress() (gas: 1862898)
[PASS] test_Initialize_RevertIf_ZeroAavePool() (gas: 1513411)
[PASS] test_Initialize_RevertIf_ZeroAdmin() (gas: 1513492)
[PASS] test_Initialize_RevertIf_ZeroManager() (gas: 1513535)
[PASS] test_Initialize_RevertIf_ZeroUpgrader() (gas: 1513459)
[PASS] test_Initialize_RevertIf_ZeroUsdc() (gas: 1513452)
[PASS] test_Initialize_RevertIf_ZeroVault() (gas: 1513396)
[PASS] test_RecoverToken_MultipleTokens() (gas: 617198)
[PASS] test_RecoverToken_RecoverAUsdc() (gas: 68504)
[PASS] test_RecoverToken_RecoverOtherToken() (gas: 517908)
[PASS] test_RecoverToken_RevertIf_NotAdmin() (gas: 56039)
[PASS] test_RecoverToken_RevertIf_ZeroRecipient() (gas: 54757)
[PASS] test_RecoverToken_Success() (gas: 69786)
[PASS] test_RecoverToken_ZeroBalance() (gas: 33686)
[PASS] test_SetVault() (gas: 65074)
[PASS] test_SetVault_RevertIf_InsufficientAllowance() (gas: 28127)
[PASS] test_SetVault_RevertIf_NotAdmin() (gas: 19868)
[PASS] test_SetVault_RevertIf_ZeroAddress() (gas: 18442)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 1437603)
[PASS] test_Upgrade_RevertIf_WithdrawerWithoutUpgraderRole() (gas: 1437614)
[PASS] test_Upgrade_Success() (gas: 1466983)
```

```
[PASS] test_Withdraw_MultipleWithdrawals() (gas: 171964)
[PASS] test_Withdraw_RevertIf_InsufficientAllowance() (gas: 47496)
[PASS] test_Withdraw_RevertIf_InsufficientBalance() (gas: 41761)
[PASS] test_Withdraw_RevertIf_MaxSentinel() (gas: 20549)
[PASS] test_Withdraw_RevertIf_NoRole() (gas: 20431)
[PASS] test_Withdraw_RevertIf_WithdrawalFailed() (gas: 150077)
[PASS] test_Withdraw_RevertIf_ZeroAddress() (gas: 18495)
[PASS] test_Withdraw_RevertIf_ZeroAmount() (gas: 20500)
[PASS] test_Withdraw_Success() (gas: 120955)
[PASS] test_Withdraw_WithPartialAllowance() (gas: 142487)
[PASS] test_Withdraw_WithPartialVaultBalance() (gas: 137629)
Suite result: ok. 46 passed; 0 failed; 0 skipped; finished in 159.77ms (233.82ms CPU time)

Ran 53 tests for test/dreRewardsDistributor.t.sol:dreRewardsDistributorTest
[PASS] testFuzz_AddRewards_AfterLateWarp_SecondSmall_ResetTo7Days_LowerRate(uint256,uint256) (runs: 256,
μ: 155390, ~: 155518)
[PASS] testFuzz_AddRewards_FirstTime(uint256) (runs: 256, μ: 112592, ~: 112741)
[PASS] testFuzz_AddRewards_Invariant_BalanceGeRewards(uint256,uint256,uint256) (runs: 256, μ: 152632, ~:
154851)
[PASS] testFuzz_AddRewards_Invariant_ETsGteCTs(uint256) (runs: 256, μ: 114004, ~: 114180)
[PASS] testFuzz_AddRewards_MultipleRounds(uint256,uint256,uint256,uint256,uint256) (runs: 256, μ: 181876,
~: 182296)
[PASS]
testFuzz_AddRewards_MultipleRounds_VestCapAndAccounting(uint256,uint256,uint256,uint256,uint256,uint256,u
int256) (runs: 256, μ: 228850, ~: 234320)
[PASS] testFuzz_AddRewards_SecondAddition(uint256,uint256,uint256) (runs: 256, μ: 161314, ~: 162056)
[PASS] testFuzz_AddRewards_ThenClaim_Accounting(uint256,uint256,uint256) (runs: 256, μ: 152868, ~:
158311)
[PASS] testFuzz_AddRewards_TwoAdds_Accounting(uint256,uint256,uint256) (runs: 256, μ: 154558, ~: 156773)
[PASS] testFuzz_AddRewards_TwoAdds_VestWindowAlwaysLeVestPeriod(uint256,uint256,uint256) (runs: 256, μ:
157750, ~: 158570)
[PASS] testFuzz_AddRewards_TwoConsecutive_SmallGap_StretchTo7Days_HigherOrSameRate(uint256,uint256)
(runs: 256, μ: 153955, ~: 155337)
[PASS] testFuzz_VestedAmount(uint256) (runs: 256, μ: 68203, ~: 68493)
[PASS] testFuzz_VestedAmount_LinearFormula(uint256,uint256,uint256,uint256) (runs: 256, μ: 159779, ~:
160865)
[PASS] test_AddRewards_After6Days_SecondSmall_StretchTo7Days_LowerRate() (gas: 142567)
[PASS] test_AddRewards_DustIncreasesRate_WhenRtsRoundsToZero() (gas: 88665)
[PASS] test_AddRewards_FirstTime() (gas: 63873)
[PASS] test_AddRewards_RevertIf_NotModerator() (gas: 18201)
[PASS] test_AddRewards_TwoConsecutiveAt1h_StretchTo7Days_HigherRate() (gas: 146390)
[PASS] test_AddRewards_WhenIdle_ResetsCTs_SoVestingStartsFromNow() (gas: 72146)
[PASS] test_AddRewards_WhenNewRewardsIsZero() (gas: 71972)
[PASS] test_AddRewards_WhenNewRewardsIsZero_ResetsSchedule_IfVestPeriodBelowThreshold() (gas: 107907)
[PASS] test_ClaimVested() (gas: 107382)
[PASS] test_ClaimVested_CanBeCalledByAnyone() (gas: 104683)
[PASS] test_ClaimVested_CappedAtDistributorBalance() (gas: 99817)
[PASS] test_ClaimVested_MultipleClaims() (gas: 118369)
[PASS] test_ClaimVested_MultipleTimesInSameBlock() (gas: 108193)
[PASS] test_ClaimVested_NoVestedRewards() (gas: 31802)
[PASS] test_ClaimVested_RevertIf_Paused() (gas: 89617)
[PASS] test_ClaimVested_WhenVaultHasNoBalance() (gas: 74580)
[PASS] test_ClaimVested_WithPartialDistributorBalance() (gas: 128415)
[PASS] test_Constructor_RevertIf_ZeroDreUSD() (gas: 39425)
[PASS] test_Constructor_RevertIf_ZeroVault() (gas: 39525)
[PASS] test_DistributorBalance() (gas: 9984)
[PASS] test_Initialize() (gas: 44911)
[PASS] test_Initialize_RevertIf_AlreadyInitialized() (gas: 22554)
[PASS] test_Initialize_RevertIf_DefaultAdminIsZeroAddress() (gas: 92767)
[PASS] test_Initialize_RevertIf_PauserIsZeroAddress() (gas: 92885)
[PASS] test_Initialize_RevertIf_UpgraderIsZeroAddress() (gas: 92877)
[PASS] test_Pause() (gas: 44098)
[PASS] test_Pause_RevertIf_NotPauser() (gas: 18206)
[PASS] test_Roles() (gas: 46088)
[PASS] test_Unpause() (gas: 33888)
[PASS] test_Unpause_RevertIf_NotPauser() (gas: 47741)
[PASS] test_Upgrade() (gas: 1192823)
[PASS] test_Upgrade_RevertIf_NotUpgrader() (gas: 1182563)
[PASS] test_VaultBalance() (gas: 9947)
[PASS] test_VaultBalance_AfterClaim() (gas: 104617)
[PASS] test_VestedAmount() (gas: 64411)
[PASS] test_VestedAmount_AfterClaim() (gas: 102306)
```

```
[PASS] test_VestedAmount_BeforeAddRewards() (gas: 14962)
[PASS] test_VestedAmount_CappedAtDistributorBalance() (gas: 60412)
[PASS] test_VestedAmount_NotInitialized() (gas: 14961)
[PASS] test_VestedAmount_WithVeryLargeTime() (gas: 60368)
Suite result: ok. 53 passed; 0 failed; 0 skipped; finished in 360.16ms (623.94ms CPU time)

Ran 1 test for test/DeployDreUSD.t.sol:DeployDreUSDTest
[PASS] test_DeployDreUSD_OnFork() (gas: 4425506)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 944.95ms (944.74ms CPU time)

Ran 2 tests for test/DeployDreSystem.t.sol:DeployDreSystemTest
[PASS] test_deployBaseComponents_DeploysAndWiresAllContracts_OnBaseSepoliaFork() (gas: 26808707)
[PASS] test_run_RevertsWhenLzEndpointNotFound() (gas: 6625)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.95s (3.85s CPU time)

Ran 18 test suites in 4.01s (6.81s CPU time): 577 tests passed, 0 failed, 0 skipped (577 total tests)
```

Code Coverage

Coverage cannot be obtained due to "Stack too deep".

Changelog

- 2026-03-02 - Initial report
- 2026-03-21 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that

Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.